

Aligning Qualitative, Real-Time, and Probabilistic Property Specification Patterns Using a Structured English Grammar

Marco Autili, Lars Grunske, Markus Lumpe, Patrizio Pelliccione, and Antony Tang, *Member, IEEE*

Abstract—Formal methods offer an effective means to assert the correctness of software systems through mathematical reasoning. However, the need to formulate system properties in a purely mathematical fashion can create *pragmatic barriers* to the application of these techniques. For this reason, Dwyer et al. invented *property specification patterns* which is a system of recurring solutions to deal with the temporal intricacies that would make the construction of reactive systems very hard otherwise. Today, property specification patterns provide general rules that help practitioners to qualify order and occurrence, to quantify time bounds, and to express probabilities of events. Nevertheless, a comprehensive framework combining qualitative, real-time, and probabilistic property specification patterns has remained elusive. The benefits of such a framework are twofold. First, it would remove the distinction between qualitative and quantitative aspects of events; and second, it would provide a structure to systematically discover new property specification patterns. In this paper, we report on such a framework and present a *unified catalogue* that combines all known plus 40 newly identified or extended patterns. We also offer a natural language front-end to map patterns to a temporal logic of choice. To demonstrate the virtue of this new framework, we applied it to a variety of industrial requirements, and use *PSPWizard*, a tool specifically developed to work with our unified pattern catalogue, to automatically render concrete instances of property specification patterns to formulae of an underlying temporal logic of choice.

Index Terms—Specification patterns, real-time properties, probabilistic properties

1 INTRODUCTION

COMPUTERIZED systems are ubiquitous and affect all aspects of modern society. However, whereas a dropped phone call or a mapping error in a language processor may just cause a minor inconvenience or difficulty, failures in components of systems that control critical functions in aircraft or industrial plants can have disastrous consequences [1]. To ensure the reliable functioning of these components we employ formal specification and verification techniques. These techniques are based on *temporal logics* and they allow us to establish the correctness of a system and its properties. For this process to work, though, we need to master temporal logics first, which is a challenging and time consuming task [2].

A temporal logic offers us a formal approach to reason about events in reactive systems. The operators of a temporal logic entail, in general, features related to the *possibility*, the *impossibility*, the *necessity*, or the *eventuality* of an event or a series of events [3]. These aspects can materialize arbitrarily interrelated in real-world systems. In addition, *order* and *occurrence* [4] of specific events may require quantification that gives rise to *real-time* [2], [5] and *probabilistic* [6] constraints that further describe the temporal relationships between different *states* of a reactive system. We can use *model checking* and other *finite-state-based reasoning* techniques to perform an automated verification of a system's compliance with given temporal properties [7], [8]. These properties are typically specified as *linear* or *branching-time* formulae [3]. Nevertheless, it remains a major challenge to capture temporal properties in a concise and correct way.

Effective system verification requires two important elements: a *model* that describes the operations and states of a system, and a *set of properties* that each implementation of this system must satisfy [1]. The specification of both need to be correct. However, correctness in this context, specifically for property specification, has different facets. In its simplest form, we just require *well-formedness* or syntactic correctness. Syntactic correctness is a necessary condition to be met prior performing verification of required system attributes, but it can easily be ensured through standard language processing techniques [9]. On the other hand, it is much harder to guarantee that a given specification matches a software engineer's intuition about the system in question. In general, the definition of the capabilities of a system and the description of its behavioral constraints have to occur at

- M. Autili is with the Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica, Università dell'Aquila, Aquila, Italy. E-mail: marco.autili@univaq.it.
- P. Pelliccione is with the Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica, Università dell'Aquila, Aquila, Italy, and is also with the Department of Computer Science and Engineering, Chalmers University of Technology, University of Gothenburg, Gothenburg, Sweden. E-mail: patrizio.pelliccione@gu.se.
- L. Grunske is with the Institute of Software Technology, University of Stuttgart, Stuttgart D-70569, Germany. E-mail: lars.grunske@informatik.uni-stuttgart.de.
- M. Lumpe and A. Tang are with the Swinburne University of Technology, Hawthorn, VIC 3122, Australia. E-mail: {mlumpe, atang}@swin.edu.au.

Manuscript received 28 Oct. 2013; revised 20 Jan. 2015; accepted 20 Jan. 2015. Date of publication 2 Feb. 2015; date of current version 17 July 2015.

Recommended for acceptance by M. Chechik.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TSE.2015.2398877

Authorized licensed use limited to: TU Delft Library. Downloaded on July 04, 2025 at 07:50:21 UTC from IEEE Xplore. Restrictions apply.

0098-5589 © 2015 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

a suitable level of abstraction and also in a way that faithfully reflects reality [10].

Nonetheless, the format and the inherent complexity of a formal specification must not interfere with desired system verification objectives. If the abstraction or formalization is too difficult to use, then we can create *pragmatic barriers* to the successful application of formal methods in practice [11]. This would naturally defeat the purpose. Dwyer et al. [4] realized this issue and developed *property specification patterns*. Patterns capture good and proven solutions to common domain problems [12], [13]. In property specification, a pattern describes a *generalized recurring system attribute* and provides a solution in form of a corresponding formal specification. Property specification patterns come with the promise of reducing the training overhead usually associated with the use of temporal logics. As a result, property specification patterns offer a means to close the *cognitive gap* between formal specification and its effective practical application in the area of property specification. Evidence for this claim and the usefulness of property specification patterns can be found among others in the banking [14], aviation [15], and automotive [16] domains.

Patterns are usually organized in catalogues. Property specification patterns are no exception [2], [4], [5], [6], [17]. According to Dwyer et al. [4], a property specification pattern catalogue comprises the “*best practices*” in system specification and represents an attempt to capture proven solutions in a single framework. However, despite their growing versatility and expressiveness, property specification patterns remain *approximations* of reality. Current catalogues describe a known *theory* of what is supposed to happen or is possible [2]. Traditionally, a *bottom-up* approach, driven by practical examples, is employed to construct pattern catalogues [13]. This technique has produced three theories, namely qualitative [4], real-time [2], [5], and probabilistic [6] patterns. Nonetheless, the associated catalogues either partly overlap or fail to account for potentially useful patterns. As a consequence, these catalogues do not offer a comprehensive and coherent system for property specification.

To address this problem, we align existing proposals for property specification patterns with the aim of defining a single, yet comprehensive and coherent, property specification pattern catalogue using the following alignment procedure:

- 1) *Literature review.* We carry out an in-depth analysis of existing approaches and patterns in the relevant literature in order to obtain the current state-of-the-art in property specification.
- 2) *Gap analysis and pattern elicitation.* Based on known modeling solutions for qualitative, real-time, and probabilistic system properties, we define a cross-tabulation of property specification patterns and catalogues. As a result, any duplications or coverage gaps are revealed and they give rise to reified or previously unknown property specification patterns and their respective representation in a suitable temporal logic.
- 3) *Validation.* We search published papers and other available documents for use cases of the newly

discovered patterns. In addition and to further strengthen our argument, we conduct an industrial case study to show the applicability of these new patterns in practice. In this way, we can demonstrate that every reified and newly found pattern warrants existence and practical application.

However, a comprehensive and coherent property specification pattern catalogue alone does not guarantee effective application in real-world scenarios. The difficulties lie in the complexity of the specification and the underlying associated mathematical techniques. Both require considerable expertise. System engineers do not always have the necessary level of training to master their use [1]. Hence, we equip our catalogue with a *natural language front-end* (i.e., a “Structured English Grammar”) to allow for a syntax-directed translation of concrete pattern instances to formulae of a temporal logic of choice. We adopt this approach from Konrad and Cheng [5] and Grunske [6] that defined a similar natural language interface to their respective catalogues. To realize this approach, we developed *Property Specification Pattern Framework* (PSPFramework) that comprises a unified pattern catalogue, a natural language interface, and a collection of associated mappings to logic formalisms. To assist system engineers, PSPFramework is equipped with PSPWizard [18], a tool to guide system engineers to express system properties through Structured English Grammar, and to translate specifications into temporal logic formulae. At present, we support several common temporal logical specification formalisms (e.g., LTL, CTL, MTL, TCTL, PLTL, and CSL) [19]. The automated translation of property specifications to temporal logic formulae enables a system engineer to focus on the system attributes rather than how these system attributes must be encoded in the target logic. This increases productivity.

The rest of the paper is organized as follows. In Section 2 we review related work and describe how specification patterns can assist software engineers to formulate system properties precisely. Section 3 presents our new unified specification pattern catalogue, including all 40 new and improved patterns resulting from this study. We discuss the core elements of PSPFramework in Section 4. In Section 5, we substantiate the effectiveness of our approach through real-world examples and provide a validation based on a dedicated industrial case study. We conclude this paper in Section 6 with a summary of our main observations and a discussion of future research directions in this area.

2 BACKGROUND AND RELATED WORK

There are three general types of property specification patterns: qualitative [4], real-time [2], [5], and probabilistic [6] specification patterns. In this section, we review their basic characteristics and highlight respective building blocks (cf. Section 2.1-2.3). Moreover, we touch upon related work on elicitation and formalization of system properties and specification pattern catalogue support, in general, in Section 2.4.

2.1 Qualitative Specification Patterns

Dwyer et al. [4] invented *property specification patterns* in the late nineties. They analyzed a set of 555 specifications from at least 35 different sources in order to define, what is now

known as, *qualitative specification patterns*.¹ Qualitative specification patterns are organized in two major groups: *occurrence patterns* and *order patterns*, comprising eight patterns in total. Occurrence patterns describe the materialization of a given event (or state) during system execution (e.g., absence or existence of an event). Order patterns, on the other hand, capture the relative sequence in which multiple events can emerge during system execution (e.g., response or precedence). It is, therefore, a catalogue that focuses solely on the modality of events. In addition, each pattern is further attributed to respect one of five generic scopes (i.e., *Globally*, *Before R*, *After Q*, *Between Q and R*, and *After Q until R*), which denote the extent of a program execution over which the pattern must hold. To complement their approach, Dwyer et al. [4] suggested feasible translations to suitable temporal logics and query languages, including LTL and CTL [3].

Dwyer et al. [4] assert that patterns belong to the community. This is evident in the ongoing efforts to use, refine, and extend them [20]. For example, Paun and Chechik [21] provided an extension that promotes events as first-class entities in property specification. Smith et al. [22] and Cabrera-Castillos et al. [23] have investigated the semantics of specification patterns when automata are used as a specification formalism. Bianculli et al. [14], on the other hand, recently investigated properties of services-based systems in the banking sector. They also argued for the inclusion of data-aware and enumerable properties specification support [24]. Finally, new specific property specification patterns have been proposed to cope with safety [25] and security concerns in safety-critical systems [26].

2.2 Real-Time Specification Patterns

If we are concerned with the design and robustness of safety-critical systems [1], [27], knowing when and in what sequence events can materialize may not be enough. We also need support to formulate constraints related to either time bounds or probabilities of events. To express real-time properties with specification patterns, Konrad and Cheng [5] amended the original specification patterns idea with a facility to denote time-based constraints in real-time temporal logics MTL [28] and TCTL [29]. The resulting *real-time specification patterns* were obtained from a detailed analysis of 80 real-time system requirements in embedded systems and automotive applications. In total, five new real-time property specification patterns organized in the categories *duration*, *periodic*, and *real-time order* were established. Like qualitative specification patterns, each real-time pattern includes a scope and workable mappings to appropriate real-time temporal logics [5]. Concurrently, Gruhn and Laue [30] developed another real-time property specification pattern catalogue that used timed observer automata as the underlying specification formalism, whereas, recently, Abid et al. [31] provided a formalization of the real-time specification patterns in *Time Transition Systems*, a generalization of *Time Petri Nets*.

A further analysis of real-time properties motivated Bellini and his colleagues to develop a *unified organization* of

specification patterns [2]. Unlike Konrad and Cheng [5] that considered real-time patterns disjoint from the original qualitative patterns, Bellini et al. sought to bring both together in a single pattern catalogue. This reorganization not only produced a uniform and consistent pattern catalogue, it also revealed a new pattern related to time-constrained precedence of events that was missing in the original model developed by Konrad and Cheng [5]. This new and unified specification pattern catalogue distinguishes between *occurrence*, *duration*, and *order* patterns. Occurrence and order patterns correspond to the original categories defined for qualitative specification patterns [4], but extended with real-time concepts. Duration patterns mimic the ones originally established for real-time specification patterns [5]. In addition, Bellini et al. [2] extended the real-time patterns with a *Time-Constrained Precedence* pattern allowing for the definition of (precedence) time ordering between events.

By joining qualitative and real-time patterns, Bellini et al. [2] completed a first step towards creating a comprehensive and coherent system for property specification. If we address probability concerns in property specification also, then we can achieve our goal of constructing a *unified catalogue* for property specification that combines all known “best practices” in a single framework.

2.3 Probabilistic Specification Patterns

Based on the earlier results on specification patterns, Grunske [6] studied probabilistic quality requirements (e.g., reliability, availability, and performance requirements) in specifications for safety-critical systems and mined those specifications for recurring *probabilistic specification patterns*. These patterns were distilled from a comprehensive literature review on probabilistic logics and probabilistic verification techniques. This study collected 154 probabilistic properties from 56 research papers [6]. The appropriateness of the defined patterns was tested on 48 quality requirements, of which 46 could be expressed in terms of a probabilistic pattern.

Probabilistic property patterns are organized in two categories: *probabilistic occurrence* and *probabilistic order*, indicating an intrinsic similarity with the original qualitative specification patterns categories [4]. However, unlike qualitative patterns, probabilistic patterns have no explicit scope. Hence, when specifying a probabilistic property, its scope is **Globally** by default. Encodings of probabilistic patterns that are contained in the catalogue are formalized in terms of *Continuous Stochastic Logic* (CSL) [32].

2.4 Elicitation and Formalization of Properties Based on Property Specification Patterns

The goal of the property specification pattern research aims at helping practitioners specify system properties correctly and easily. To support this idea, tools and concepts [2], [4], [5], [6] have been made available to effectively guide users to select an appropriate property specification pattern. For example, Konrad and Cheng [5] and Grunske [6] accompanied their respective pattern catalogue with a *Structured English Grammar* with the intent to facilitate property specification through a natural language interface. On the other hand, PROPEL (PROPErty ELucidator) [22] and its

1. Available online at: <http://patterns.projects.cis.ksu.edu>

Authorized licensed use limited to: TU Delft Library. Downloaded on July 04, 2025 at 07:50:21 UTC from IEEE Xplore. Restrictions apply.

extension [33] employ *Question Trees* to navigate through the process of producing formal property specifications. The *Question Tree*-based approach requires system engineers to answer a series of questions to derive both a natural language description and a finite-state automaton for a property of interest. Originally, Smith et al. [22] required that a system engineer had to know and select the specific pattern template before defining the actual property. Cobleigh et al. [33] revised this approach by posing additional questions to the system engineer in order to help selecting a template. Although PROPEL focuses only on qualitative property specification, its approach could potentially serve as a basis for real-time and probabilistic property specification.

Another technique is *Wizard for Property Sequence Charts* (WPSC) [34], [35] that assists system engineers in resolving potential occurrences of uncertainty and ambiguity in system requirements. These issues give rise to inconsistencies between requirements and, hence, would create major challenges for all stakeholders of a software solution [36]. WPSC uses a set of guiding sentences (hints) derived from the expertise of engineers in requirements formalization to guide software architects towards a proper selection of qualitative specification patterns [4]. However, WPSC lacks support for timed and probabilistic properties [37], [38].

Finally, DDPSL [39] is a tool to ease the description of event sequences or relations between events in a subset of *Property Specification Language* (PSL) [40]. PSL is a formal language especially designed for capturing the qualitative properties of electronic systems. It includes as sublanguages LTL and CTL. However, its inherent complexity is extremely demanding and requires well-trained engineers to define the properties correctly [39]. Similar to WPSC, DDPSL does not support the specification of timed and probabilistic aspects of system requirements.

3 A COORDINATED VIEW OF PROPERTY SPECIFICATION PATTERNS

Previous work in specifications patterns primarily focused on a specific aspect of system properties, which resulted in three yet partly disjoint pattern catalogues for the specification of qualitative, runtime, and probabilistic system properties. Using a top-down approach, we applied cross-tabulation and gap analysis to the existing pattern catalogues. As a result, we were able to infer and define new patterns that provide additional solutions to property specification problems [20]. Our investigation resulted in 40 new or extended patterns. These new or extended patterns, previously not found with traditional bottom-up approaches, address real-world system properties. A validation, described in Section 5, provides further evidence for the appropriateness of these new or extended patterns.

An attentive reader might notice that some newly discovered patterns can be achieved by composing existing scopes and patterns. A dedicated specification pattern is generally intuitive and straightforward to use. On the contrary, specifying properties with a combination of patterns can be complex and intricate. For instance, an engineer may require a combination of tricky instantiations intermixing scopes and patterns in order to specify required properties. The framework presented in this paper

introduces new patterns as first class representations to provide ready-to-use solutions with direct mappings into temporal logic formulae. This approach can minimize errors during the specification process. The tool supporting the framework, described in Section 4.3 espouses this idea and guides the user to the definition of desired properties through proper instantiation of patterns and scopes.

Table 1 provides a summary (*i.e.*, a cross-tabulation) of the specification patterns analyzed in this work. For each pattern, we give its name and corresponding catalogue (*i.e.*, context) in which it can materialize. The initial cross-tabulation revealed inherent similarities across different pattern catalogues and made existing coverage gaps explicit. An in-depth gap analysis resulted in three possible outcomes: (i) *new*—a newly discovered pattern, (ii) *N/A*—a pattern is not applicable in this context, and (iii) *extended*—a pattern, originally defined for scope **Globally** only [6], has been amended to fit non-global scopes as well.

We obtained the outcomes by coordinating existing pattern catalogues and using our alignment procedure (*i.e.*, literature review, gap analysis and pattern elicitation, and validation). Fig. 1 shows our coordinated view of specification patterns, highlighting the qualitative patterns introduced by Dwyer et al. [4], the real-time patterns developed by Konrad and Cheng [5] and Bellini et al. [2], the probabilistic patterns suggested by Grunske [6], and finally the new patterns discovered in this work. We also include *inter-catalogue* and *intra-catalogue inheritance relations* in this figure. An intra-catalogue relation represents a correspondence between patterns that belong to the same catalogue. An example is *Precedence* and *Precedence with 1-cause and N-effects*. Both patterns specify qualitative system properties but the latter is a generalization of the former. An inter-catalogue inheritance relation signifies that a given pattern can be found in more than one category. For example, the *Existence* pattern defines an inter-catalogue inheritance as it can occur in a qualitative, a real-time, a probabilistic context, or a combination thereof.

Occurrence patterns—According to Dwyer et al. [4] occurrence patterns detail the circumstances for a given event/state to arise during system execution.

- *Existence* (also known as *Eventually* or *Future*) describes the extent of a system's execution containing specified events or states. This pattern also spawns *Bounded Existence*, defined within the qualitative specification pattern catalogue, to denote that a given event/state can only occur a fixed number of times. Consequently, these two patterns are connected through an intra-catalogue relation (cf. Fig. 1). We added *Time-constrained Existence*, an inter-catalogue extension, to bridge the gap between its qualitative and probabilistic forms. The latter is specialized by the *Transient State Probability* pattern, which allows for the specification of a system property in which a given event/state holds with a probability bound $\bowtie p$ exactly after t time units, where $p \in [0, 1]$ is a real value as a probability threshold and $\bowtie \in \{\geq, >, <, \leq\}$ is a comparison operator associated with this probability threshold. A time-constrained extension to *Bounded Existence* appears not warranted as there is no

TABLE 1
Pattern Overview

Pattern Name	Qualitative	Real-time	Probabilistic
Absence	[4]	new	[6] / new ^{1,2,3}
Universality	[4]	new	[6] / new ^{1,2,3}
Existence	[4]	new	[6] / extended ³
Bounded Existence	[4]	N/A	N/A
Transient State Probability	N/A	N/A	[6] / extended ³
Steady State Probability	N/A	N/A	[6] / extended ³
Minimum Duration	N/A	[5]	new
Maximum Duration	N/A	[5]	new
Recurrence	new	[5] / extended ^{4,5}	new
Precedence	[4]	[2]	[6] / extended ³
Precedence Chain _{N-1}	[4]	new ⁷	new ⁷
Precedence Chain _{1-N}	[4]	new ⁷	new ⁷
Constrained Precedence Chain _{N-1}	[4]	new ⁷	new ⁷
Constrained Precedence Chain _{1-N}	[4]	new ⁷	new ⁷
Response	[4]	[5] ⁴	[6] / extended ^{3,4}
Response Chain _{N-1}	[4]	new ⁶	new ⁶
Response Chain _{1-N}	[4]	new ⁶	new ⁶
Constrained Response	new	new	[6] / extended ³
Constrained Response Chain _{N-1}	[4]	new ⁶	new ⁶
Constrained Response Chain _{1-N}	[4]	new ⁶	new ⁶
Response Invariance	new	[5] / extended ⁴	new
Until	new	new	[6] / extended ³

Notes: ¹ For the upper-bounded scopes “Before R ”, “Between Q and R ”, and “After Q until R ” we provided one possible interpretation, namely “until the end of the time bound” or R , whatever occurs first. ² This pattern is known as *Probabilistic Invariance*. ³ The pattern has been extended for non global scopes. ⁴ The pattern has been extended to allow for the specification of time intervals. ⁵ This pattern is known as *Bounded Recurrence*. ⁶ For branching time logics only specification with a global scope are available. ⁷ Not available for branching time logics.

evidence, in published papers and other available documents, for the need of such a variant. The same applies to potential probabilistic support, as the likelihood of an event happening is not definitive.

- *Absence* (also known as Never) describes the extent of a system’s execution that is free of a specific event/state. To amend the qualitative version, we added corresponding real-time and probabilistic pattern

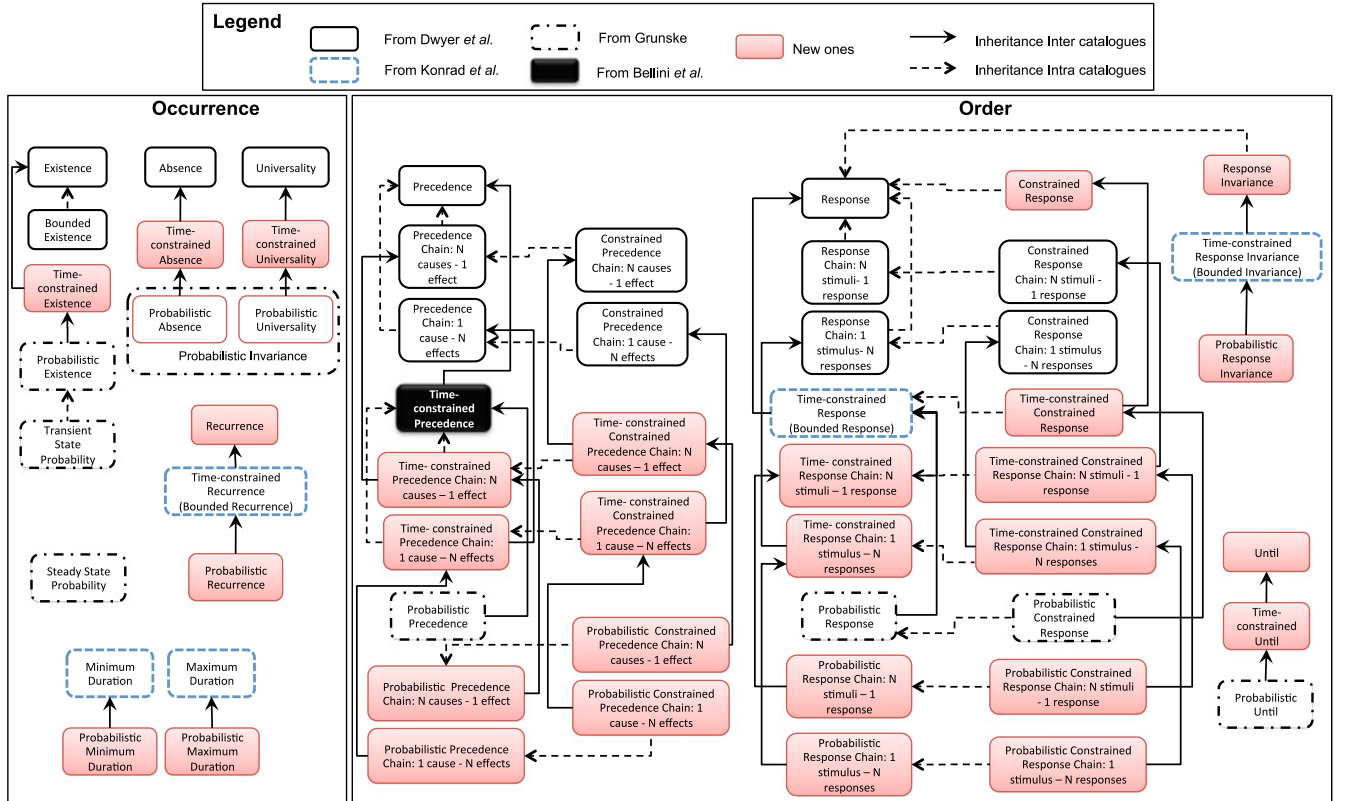


Fig. 1. Specification patterns: a coordinated view.

variants to account for the eventual presence of quantitative constraints.

- *Universality* (also known as Globally, Henceforth, or Always) traces the extent of a system's execution which contains only a desired event/state. Similarly to the *Absence* pattern, we added a time-constrained and a probabilistic version. The *Universality* pattern is a close cousin of the *Absence* pattern. The universality of an event/state is just the negation of its absence. The authors in [6] joins *Probabilistic Absence* and *Probabilistic Universality* to become *Probabilistic Invariance*.
- *Recurrence* has existed solely in terms of a real-time pattern to denote "the amount of time in which a state formula has to hold at least once" [5]. We also identified viable qualitative and probabilistic forms. The qualitative version captures the extent of a system's execution that must repeatedly contain a specific event/state. As pointed out by [5], the recurrence pattern surfaces in many publications, including [41], and it is commonly associated with the absence of non-progress cycles in a system. The probabilistic variant, on the other hand, describes the periodic satisfaction of a given event/state. Intuitively, it captures a property requiring that, in a given interval, a designated event/state has to hold at least once.
- *Minimum/Maximum Duration* allow for expressing a real-time scenario in which every time a specific event/state occurs (i.e., a state formula switches between false and true), it will remain true for at least (or at most) t time units. We extended these patterns with probabilistic versions, as a corresponding property can be subject to a probability bound also.
- *Steady state* requires that, in the long run, a given event/state must hold with a specified probability bound. This property is probability-specific and has no bearing on qualitative or real-time property specifications.

Order patterns—Following [4], order patterns describe the relative sequences in which multiple events/states can occur during system execution.

- *Precedence* captures a basic ordering relationship between a pair of events/states, where the occurrence of the first event/state is a necessary pre-condition for the occurrence of the second. In other words, the second event/state is enabled by the occurrence of the first one. The qualitative version of this pattern is due to [4], [2] offered a time-constrained variant, whereas [6] identified a corresponding probabilistic form.
- *Precedence Chain* is a generalization of the *Precedence* pattern and entails a more complex relationship between a series of events/states. Originally, [4] identified two principle qualitative variants: (i) *N-causes and 1-effect (N-1)* to express that a given dependent event/state must have been preceded by a sequence of causal events/states, and (ii) *1-cause and N-effects (1-N)* to indicate that a sequence of dependent events/states must have been preceded by a singly-designated causal event/state within the same scope. We added time-constrained and

probabilistic patterns to align them with their qualitative instances.

The authors in [4] suggested also a constrained variant of the qualitative *Precedence Chain* pattern to express that some events/states are not to occur within a chain. We added in our framework viable forms for real-time and probabilistic specifications, named *Time-constrained Constrained Precedence Chain*, and *Probabilistic Constrained Precedence Chain* for both sub-types *N-1* and *1-N*.

- *Response* (also known as Follows or Leads-to) describes that a primary causal event/state (i.e., the stimulus) must be followed by a secondary dependent event/state (i.e., the response). There appears to be a strong connection between the *Precedence* and the *Response* patterns. However, they are very different [4]. *Response* allows effects to occur without causes, whereas *Precedence* permits causes without subsequent effects. A qualitative version of this pattern was developed by [4], the time-constrained one is due to [5], and [6] proposed a probabilistic variant.

Konrad and Cheng [5] defined also another type of response pattern, called *Bounded Invariance*, that allows for the specification of a minimum time span an event/state must hold once another event/state has occurred. This pattern captures a situation in which the occurrence of an initial stimulus must be followed by an *invariant* occurrence of a corresponding response (i.e., the response must hold continually). We have identified viable qualitative and probabilistic variants also. In particular, we added a new (qualitative) *Bounded Invariance* and a *Probabilistic Response Invariance* pattern, and renamed the original real-time pattern defined by [5] to *Time-constrained Response Invariance*.

Finally, Grunske [6] identified a further means to denote the absence of a third event/state within the *Response* pattern. As a result, Grunske [6] formulated the *Probabilistic Constrained Response* pattern to express that an occurrence of an event/state (the stimulus) must be followed by the occurrence of a secondary event/state (the response) without a third event/state (the constraint) ever holding within a given time and probability bound. We follow this proposal and added respective qualitative and time-constrained versions, namely *Constrained Response* and *Time-constrained Constrained Response*.

- *Response Chain* is again a generalization of the *Response* pattern and captures the more complex relationship between a series of causal (i.e., the stimuli) and dependent (i.e., responses) events/states. The two principle qualitative variants [4] are: (i) *N-stimuli and 1-response (N-1)* to express that a given dependent event/state has to occur in response to a causal sequence of primary events/states (the stimuli), and (ii) *1-stimulus and N-responses (1-N)* to signify that a sequence of dependent secondary events/states (the responses) has to materialize after a causal primary event/state (the stimulus) within the same scope. We added a time-constrained and a probabilistic version of *Response Chain*.

Dwyer et al. [4] suggested also a constrained variant of the qualitative *Response Chain* pattern to express that some events/states are not to occur within a chain. There are viable forms for real-time and probabilistic specifications too. As a result, we included in our framework *Constrained Response Chain*, *Time-constrained Constrained Response Chain*, and *Probabilistic Constrained Response Chain* for both sub-types *N-1* and *1-N*.

- *Until* has been proposed by [6]. This pattern describes a scenario in which an event/state will eventually become true within a given time bound, after another event/state held continuously with a specified probability bound. We added a qualitative and a time-constrained version of *Until*.

To corroborate that a top-down approach can indeed yield viable new specification patterns, consider, for example, *Time-constrained Precedence Chain* (*Precedence* and *Response Chain* patterns are among the most interesting newly identified patterns). A practical scenario in which to apply *Time-constrained Precedence Chain* with 2 causes and 1 effect (i.e., a specialization of *Time-constrained Precedence Chain N-causes and 1-effect*) is the specification of the behavior of an *anti-lock braking system*.²

An anti-lock braking system (ABS) system involves a *brake assist system*, which is activated only when, no more than 50 ms before, the driver was attempting a “panic stop” (by detecting that the brake pedal was depressed very fast, unlike a normal stop where the pedal pressure would usually be gradually increased). A panic stop occurs when the electronic control unit (ECU) detects a wheel rotating significantly slower than the others (*first cause*) and afterwards the valves have been actuated to reduce hydraulic pressure to the brake at the affected wheel (*second cause*). Then, as the effect of these two causes, the brake assist system increases braking force where not enough pressure is applied.

A globally-scoped temporal logical solution of the *Time-constrained Precedence Chain* pattern in MTL is:

$$\Box(\Diamond^{[t_u^P, t_u^P]} P \rightarrow (\Diamond^{[0, t_u^P - t_l^P]} (S \wedge \bigcirc(\Diamond^{[0, t_u^P - t_l^P - t_u^T]} T))))),$$

where t_u^P and t_u^T are the upper time bounds for the events P and T , respectively, and t_l^P is the lower time bound for P . In this example t_u^P is 50 ms, whereas t_l^P and t_u^T have been omitted. Therefore, the final formula becomes:

$$\Box(\Diamond^{[50, 50]} P \rightarrow (\Diamond^{[0, 50]} (S \wedge \bigcirc(\Diamond^{[0, 50]} T))))$$

with $P = \{\text{the brake assist system increases braking force where not enough pressure is applied}\}$, $S = \{\text{the electronic control unit (ECU) detects a wheel rotating significantly slower than the others}\}$, and $T = \{\text{the valves have been actuated to reduce hydraulic pressure to the brake at the affected wheel}\}$.

In this formula, for every occurrence of P , the time interval of 50 ms immediately preceding the considered occurrence of P is checked. The intuition of the precedence chain pattern is to look for the occurrences of two consecutive events S and T once event P has occurred within 50 ms.

2. Available online at: http://en.wikipedia.org/wiki/Anti-lock_braking_system.

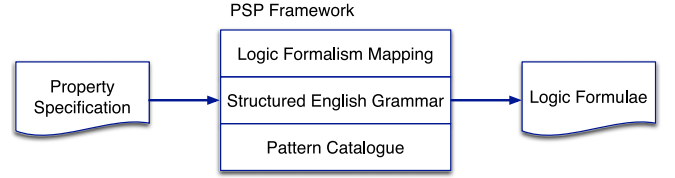


Fig. 2. Structure of the PSPFramework.

Consequently, $\Diamond^{[50, 50]} P$ is the precondition of the implication and the effect of the time-constrained precedence chain. If this precondition is true, then the formula requires that within the time interval $[0, 50]$ the two causes must have occurred (i.e., there is an occurrence of S followed by an occurrence of T prior P). In other words, the formula is satisfied, if within time interval $[0, 50]$ eventually S occurs and afterwards, within the same time interval, an occurrence of T takes place. On the other hand, if this precondition is false, then the formula is evaluated to true.

4 PSPFRAMEWORK: A PROPERTY SPECIFICATION PATTERN FRAMEWORK

The primary goal of this work is to simplify the specification of system properties. To this end, we created *PSPFramework*, a three-tier approach consisting of a unified pattern catalogue, a natural language interface based on Structured English Grammar, and a collection of mappings to suitable logic formalisms. Fig. 2 depicts the structure of the *PSPFramework*. The pattern catalogue in the bottom tier defines all the patterns that we have catalogued in this work (cf. Table 1).

The Structured English Grammar, an English-like specification scheme to capture pattern-based system properties, constitutes the middle tier. Conceptually, the Structured English Grammar is paired with a set of logic formalism mappings. There exists an isomorphism between each Structured English Grammar phrase and a corresponding formula in the underlying logic formalism of choice. The Structured English Grammar together with corresponding isomorphisms yield a set of *syntax-directed* definitions [9] that system engineers can use to formalize requirements (i.e., system properties) in a pattern-based fashion.

The top tier encompasses supported pattern mappings to logic formalisms of choice. At present, it provides support for LTL, CTL, MTL, TCTL, CSL, and PLTL [19]. *PSPFramework* contains qualitative, real-time, and probabilistic property specification patterns. However, a given logic formalism may only offer limited support for representing a specific pattern instance (i.e., qualitative, real-time, or probabilistic version). As a result, some patterns may not be denotable in the logic formalism of choice. We can express this by omitting a corresponding mapping from a Structured English Grammar phrase (or sub-phrase) to the target logic (cf. [19]). In general, qualitative patterns are mapped to LTL or CTL, real-time patterns to MTL or TCTL, and probabilistic patterns to CSL or PLTL [19]. In this paper, we present the mapping of Structured English Grammar to MTL in order to demonstrate the functioning of *PSPFramework*. We refer the reader to the wiki web-pages [19] for information on mappings to the other logic formalisms.

In summary, *PSPFramework* fosters a specification writing process that not only assists system engineers in

TABLE 2
Structured English Grammar

$Q, P, R, S, T, Z \in \text{Events/States}$ $t_l^P, t_u^P \in \mathbb{R}^+$, where P is the associated event and t_u, t_l are the upper and lower time bounds, respectively $p \in \mathbb{R}^{[0,1]}$, where p is a probability bound $n \in \mathbb{Z}$	
Property	::= Scope, Pattern.
Scope	::= Globally Before $\{R\}$ After $\{Q\}$ Between $\{Q\}$ and $\{R\}$ After $\{Q\}$ until $\{R\}$
Pattern	::= Occurrence Order
Occurrence	::= Universality Absence Existence BoundedExistence TransientState SteadyState MinimumDuration MaximumDuration Recurrence
Universality	::= it is always the case that $\{P\}$ [holds] $[\text{Time}(P)]$ $[\text{Probability}]$
Absence	::= it is never the case that $\{P\}$ [holds] $[\text{Time}(P)]$ $[\text{Probability}]$
Existence	::= $\{P\}$ [holds] eventually $[\text{Time}(P)]$ $[\text{Probability}]$
BoundedExistence	::= $\{P\}$ [holds] at most n times $[\text{Time}(P)]$ $[\text{Probability}]$
TransientState	::= $\{P\}$ [holds] after t_u^P TimeUnits $[\text{Probability}]$
SteadyState	::= $\{P\}$ [holds] in the long run $[\text{Probability}]$
MinimumDuration	::= once $\{P\}$ [becomes satisfied] it remains so for at least t_u^P TimeUnits $[\text{Probability}]$
MaximumDuration	::= once $\{P\}$ [becomes satisfied] it remains so for less than t_u^P TimeUnits $[\text{Probability}]$
Recurrence	::= $\{P\}$ [holds] repeatedly [every t_u^P TimeUnits] $[\text{Probability}]$
Order	::= Precedence PrecedenceChain _{1N} PrecedenceChain _{N1} Until Response ResponseChain _{1N} ResponseChain _{N1} ResponseInvariance
Precedence	::= if $\{P\}$ [holds] then it must have been the case that $\{S\}$ [has occurred] $[\text{Interval}(P)]$ before $\{P\}$ [holds] $[\text{Probability}]$
PrecedenceChain _{1N}	::= if $\{S\}$ [has occurred] and afterwards $(\{T_i\} [\text{UpperTimeBound}(T_i)] [\text{Constraint}(T_i)])^{(1 \leq i \leq N-1; ",")}$ [hold] then it must have been the case that $\{P\}$ [has occurred] $[\text{Interval}(S)]$ before $\{S\}$ [holds] $[\text{Constraint}(S)]$ $[\text{Probability}]$
PrecedenceChain _{N1}	::= if $\{P\}$ [holds] then it must have been the case that $\{S\}$ and afterwards $(\{T_i\} [\text{UpperTimeBound}(T_i)] [\text{Constraint}(T_i)])^{(1 \leq i \leq N-1; ",")}$ [have occurred] $[\text{Interval}(P)]$ before $\{P\}$ [holds] $[\text{Probability}]$
Until	::= $\{P\}$ [holds] without interruption until $\{S\}$ [holds] $[\text{Time}(P)]$ $[\text{Probability}]$
Response	::= if $\{P\}$ [has occurred] then in response $\{S\}$ [eventually holds] $[\text{Time}(S)]$ $[\text{Constraint}(S)]$ $[\text{Probability}]$
ResponseChain _{1N}	::= if $\{P\}$ [has occurred] then in response $\{S\}$ [eventually holds] $[\text{Time}(S)]$ $[\text{Constraint}(S)]$ followed by $(\{T_i\} [\text{Time}(T_i)] [\text{Constraint}(T_i)])^{(1 \leq i \leq N-1; ",")}$ [eventually hold] $[\text{Probability}]$
ResponseChain _{N1}	::= if $\{S\}$ followed by $(\{T_i\} [\text{Time}(T_i)] [\text{Constraint}(T_i)])^{(1 \leq i \leq N-1; ",")}$ [have occurred] then in response $\{P\}$ [eventually holds] $[\text{Time}(P)]$ $[\text{Constraint}(P)]$ $[\text{Probability}]$
ResponseInvariance	::= if $\{P\}$ [has occurred] then in response $\{S\}$ [holds] continually $[\text{Time}(S)]$ $[\text{Probability}]$
Constraint(P)	::= without $\{Z^P\}$ [holding] in between
Time(P)	::= UpperTimeBound(P) LowerTimeBound(P) Interval(P)
UpperTimeBound(P)	::= within t_u^P TimeUnits
LowerTimeBound(P)	::= after t_l^P TimeUnits
Interval(P)	::= between t_l^P and t_u^P TimeUnits
TimeUnits	::= any denomination of time (e.g., seconds, minutes, hours, days, or years)
Probability	::= with a probability (lower than p lower or equal than p greater than p greater or equal than p)

capturing properties in a concise and correct manner, but also provides the means to perform property specifications at the right level of abstraction. More precisely, when a system engineer begins to define a system property, they can do so in a top-down fashion starting with the most general and abstract property category first. Once an initial property structure has been elected, they can refine the property by focusing on specific subordinate attributes (e.g., time or probability bounds). Traditionally [5], [6], a system engineer has to commit to a specific category (i.e., qualitative, real-time, or probabilistic specification) as the first action.

4.1 Structured English Grammar

Mathematical logic, though the most effective means to systematically capture system properties, is, in general, too strict and most software engineers prefer natural language to formulate their system requirements [5], [6]. On the other hand, a purely natural language may be too weak and error-prone, as it lacks the rigor required to effectively capture system properties. A Structured English Grammar, as the interface to property specification patterns, addresses this shortcoming [5], [6]. Structured English Grammar utilizes basic English phrases, but limits their composition to terms denotable in temporal logics. As a result, Structured English Grammar helps to bridge the conceptual gap

between the extremes on either side: natural language specification and pure mathematical reasoning.

The syntax of our Structured English Grammar is shown in Table 2. In the grammar, terminal strings are written in **bold** font, whereas non-terminal symbols are given in true type font. We use $[\]$ to denote an optional symbol. For example, we write **[holds]** to express that the English verb “holds” can be omitted from a concrete pattern specification, if the resulting sentence is closer to natural language. Additionally, we use curly brackets $\{ \}$ to separate the description of an event/state from the enclosing pattern structure and write $\{P\}$ to denote the occurrence of an event/state P .

The grammar’s general format follows the one proposed by Konrad and Cheng [5] and Grunske [6] with one exception: we use an *attribute grammar* [9]. Attribute grammars allow for the specification of *inherited attributes* that we can use to capture dependencies between different parts of a phrase. Such dependencies arise when defining desired time bounds and event constraints. The inherited attributes in our grammar are resolved with respect to the event a given time bound or constraint specification relates to. For instance, we write $\text{Time}(P)$ to denote the time bound being associated with an event/state P . Consequently, the notation for the upper and lower time bound are t_u^P and t_l^P ,

respectively. For chains, we use subscript i , $1 \leq i \leq N - 1$, to indicate the i th event/state in a set of events/states $T_1, \dots, T_{N-1}$. Constraints specification follow the approach used for time bounds. A constraint linked to an event/state P is expressed by the term $\text{Constraint}(P)$, and we use Z^P to mean the actual constraint Z associated with event/state P .

To illustrate the use of Structured English Grammar, consider the following property specification:

“Globally,

if { *a sensor detects that a train is less than 800 m from a level-crossing and the boomgate is up* }
then in response { *a red warning light flashes* } **within**
 2 seconds **followed by** { *the boomgate is being lowered* } **within 5 seconds.”**

This is an instance of $\text{ResponseChain}_{1-N}$, more precisely the real-time *Response Chain 1-stimulus and 2-responses*. The corresponding phrase in the Structured English Grammar (cf. Table 2) contains $\text{Time}(S)$ and $\text{Time}(T_i)$, $1 \leq i \leq N - 1$, as means to denote real-time bounds. $\text{Time}(S)$ captures the maximum delay between the event $P = \{a \text{ sensor detects that a train is less than 800m from a level-crossing and the boomgate is up}\}$, the stimulus, and $S = \{a \text{ red warning light flashes}\}$, namely $\text{Time}(S) = \text{“within 2 seconds.”}$ The time bound $\text{Time}(T_1) = \text{“within 5 seconds”}$ for event $T_1 = \{the \text{ boomgate is being lowered}\}$ specifies the maximum distance, in seconds, between the occurrences of the responses S and T_1 .

Each sentence derivable from our Structured English Grammar serves as an instantiation of either a qualitative, real-time, and probabilistic pattern or a combination thereof. The start symbol of the grammar is **Property** and the number of possible pattern phrases stemming from it is *finite* by design. To illustrate the derivation process, that is, how our grammar supports pattern-based specification of properties consider, for example, the following property:

“After a write operation, the memory location should retain the data for the next 3 years with a probability of 99.9 percent.”

This requirement is an instance of the $\text{ResponseInvariance}$ pattern with $P = \{a \text{ write operation}\}$, $S = \{the \text{ memory location retains the data}\}$, an upper time bound $\text{Time}(S) = \text{“3 years”}$, and a probability bound 0.999. The following derivation sequence illustrates how to generate the corresponding specification pattern sentence:

Property
 $\Rightarrow$ **Scope, Pattern.**
 $\Rightarrow$ **Globally, ResponseInvariance.**
 $\Rightarrow$ **Globally, if** { P } **[has occurred]** **then in response** { S }
[holds] continually [Time(S)] [Probability].
 $\Rightarrow$ **Globally, if** { *a write operation* } **has occurred then in response** { *the memory location retains the data* } **continually within 3 years with a probability greater or equal than 0.999.**

This sentence can now be mapped to a suitable logic formalism. The target language must support both time bounds and probability bounds (e.g., PLTL or CSL).

Finally, the Structured English Grammar allows system engineers to specify the behavior of a system for the

purpose of model checking in a way that would otherwise be difficult, if not impossible, due to the complexity of logical constructs. However, to be useful it must be unambiguous [9]. More precisely, the formation of the Structured English Grammar must not produce more than one derivation sequence for a given pattern instance. In other words, the grammar must be either $\text{LL}(k)$ or $\text{LR}(k)$ for some k [9]. Our grammar satisfies this property. In addition, it can be transformed to an equivalent $\text{LL}(1)$ grammar to facilitate implementation, as, for example, in PSPWizard [18].

4.2 Mapping Pattern Instances to Temporal Logic Formulae

The phrases generated by the Structured English Grammar embody specific instances of a qualitative, real-time, or probabilistic pattern. In addition, each pattern can occur within one of five generic scopes. These scopes are an integral component of a pattern and, hence, cannot be considered in isolation. More precisely, when mapping a given pattern instance, we need to factor in its associated scope. As a result, a concrete mapping of a given pattern to a suitable underlying logic formalism varies depending on the scope within which the pattern occurs. In other words, for each pattern there are five distinct encodings to an underlying logic formalism, and these encodings are parameterized over the corresponding scope expression.

The mapping of our pattern catalogue to *Metric Temporal Logic* (MTL) [28] is shown for occurrence patterns in Table 3 and for order patterns in Tables 4 and 5. A mapping rule entails three components: the pattern name, a scope, and a scope-specific logic formula template. The logic formula templates create a term rewriting system that uses a set of attribute evaluation functions $\llbracket \cdot \rrbracket$ to inject specified time and probability bounds into the target formula. Table 6 provides a summary of all required evaluation functions for PSPFramework. The evaluation functions are the same for all supported logic formalism. However, if a specific formalism does not support an attribute in question, then the evaluation functions maps this attribute to an empty string.

To illustrate the mapping process, consider the following sentence:

“Globally, if { *a car stops at the left-turn lane* } **then in response** { *the filter traffic light will turn green* } **within 60 seconds.”**

The MTL formula of this instance of a *Response* pattern occurring in scope **Globally**, with $P = \{a \text{ car stops at the left-turn lane}\}$, $S = \{the \text{ filter traffic light will turn green}\}$, and $\llbracket \text{umintime}(S) \rrbracket = [0, 60]$, is:

$$\Box ((a \text{ car stops at the left-turn lane}) \rightarrow \Diamond^{[0,60]} (the \text{ filter traffic light will turn green})).$$

The mapping process for LTL, CTL, TCTL, CSL, and PLTL follows the same principles. Table 7 illustrates this for the *Universality* pattern.

4.3 PSPWizard: A Tool to Specify Temporal Logical Properties with Specification Patterns

Pairing the Structured English Grammar with a target logic is not an easy task. Moreover, the capabilities of specific

TABLE 3
Occurrence Patterns Mapping to Metric Temporal Logic

Absence:	
Globally	$::= \Box \llbracket \text{time}(P) \rrbracket \neg P$
Before {R}	$::= \Diamond \llbracket \text{mintime}(P) \rrbracket R \rightarrow (\neg P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R \vee \Box \llbracket \text{time}(P) \rrbracket \neg P)$
After {Q}	$::= \Box (Q \rightarrow \Box \llbracket \text{time}(P) \rrbracket \neg P)$
Between {Q} and {R}	$::= \Box ((Q \wedge \Box \llbracket \text{umintime}(P) \rrbracket \neg R) \wedge \Diamond \llbracket \text{mintime}(P) \rrbracket R) \rightarrow (\neg P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R \vee \Box \llbracket \text{time}(P) \rrbracket \neg P)$
After {Q} until {R}	$::= \Box ((Q \wedge \Box \llbracket \text{umintime}(P) \rrbracket \neg R) \rightarrow \neg P \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} R)$
Universality:	
Globally	$::= \Box \llbracket \text{time}(P) \rrbracket P$
Before {R}	$::= \Diamond \llbracket \text{mintime}(P) \rrbracket R \rightarrow (P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R \vee \Box \llbracket \text{time}(P) \rrbracket P)$
After {Q}	$::= \Box (Q \rightarrow \Box \llbracket \text{time}(P) \rrbracket P)$
Between {Q} and {R}	$::= \Box ((Q \wedge \Box \llbracket \text{umintime}(P) \rrbracket \neg R) \wedge \Diamond \llbracket \text{mintime}(P) \rrbracket R) \rightarrow (P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R \vee \Box \llbracket \text{time}(P) \rrbracket P)$
After {Q} until {R}	$::= \Box ((Q \wedge \Box \llbracket \text{umintime}(P) \rrbracket \neg R) \rightarrow P \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} R)$
Existence:	
Globally	$::= \Diamond \llbracket \text{time}(P) \rrbracket P$
Before {R}	$::= \neg R \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} (P \wedge \neg R)$
After {Q}	$::= \Box \neg Q \vee \Diamond (Q \wedge \Diamond \llbracket \text{time}(P) \rrbracket P)$
Between {Q} and {R}	$::= \Box ((Q \wedge \Box \llbracket \text{umintime}(P) \rrbracket \neg R) \wedge \Diamond \llbracket \text{mintime}(P) \rrbracket R) \rightarrow (\neg R \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} (P \wedge \neg R))$
After {Q} until {R}	$::= \Box ((Q \wedge \Box \llbracket \text{umintime}(P) \rrbracket \neg R) \rightarrow (\neg R \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} (P \wedge \neg R)))$
BoundedExistence:	
N/A	
TransientState:	
N/A	
SteadyState:	
N/A	
MinimumDuration:	
Globally	$::= \Box (P \vee (\neg P \mathcal{W}^{\llbracket \text{utb}(P) \rrbracket} P))$
Before {R}	$::= \Diamond R \rightarrow ((P \vee (\neg P \mathcal{U}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \neg R) \vee R)) \mathcal{U} R)$
After {Q}	$::= \Box (Q \rightarrow \Box (P \vee (\neg P \mathcal{W}^{\llbracket \text{utb}(P) \rrbracket} P)))$
Between {Q} and {R}	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((P \vee (\neg P \mathcal{U}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \neg R) \vee R)) \mathcal{U} R))$
After {Q} until {R}	$::= \Box ((Q \wedge \neg R) \rightarrow ((P \vee (\neg P \mathcal{W}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \neg R) \vee R)) \mathcal{W} R))$
MaximumDuration:	
Globally	$::= \Box (P \vee (\neg P \mathcal{W}^{\llbracket \text{utb}(P) \rrbracket} \neg P))$
Before {R}	$::= \Diamond R \rightarrow ((P \vee (\neg P \mathcal{U}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \Diamond \llbracket \text{utb}(P) \rrbracket} (\neg P \wedge R) \vee R)) \mathcal{U} R)$
After {Q}	$::= \Box (Q \rightarrow \Box (P \vee (\neg P \mathcal{W}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \Diamond \llbracket \text{utb}(P) \rrbracket} \neg P))))$
Between {Q} and {R}	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((P \vee (\neg P \mathcal{U}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \Diamond \llbracket \text{utb}(P) \rrbracket} (\neg P \wedge R) \vee R)) \mathcal{U} R))$
After {Q} until {R}	$::= \Box ((Q \wedge \neg R) \rightarrow ((P \vee (\neg P \mathcal{W}^{\llbracket \text{utb}(P) \rrbracket} (P \wedge \Diamond \llbracket \text{utb}(P) \rrbracket} (\neg P \wedge R) \vee R)) \mathcal{W} R))$
Recurrence:	
Globally	$::= \Box (\Diamond \llbracket \text{utb}(P) \rrbracket P)$
Before {R}	$::= \Diamond R \rightarrow ((\Diamond \llbracket \text{utb}(P) \rrbracket (P \vee R)) \mathcal{U} R)$
After {Q}	$::= \Box (Q \rightarrow \Box (\Diamond \llbracket \text{utb}(P) \rrbracket P))$
Between {Q} and {R}	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((\Diamond \llbracket \text{utb}(P) \rrbracket (P \vee R)) \mathcal{U} R))$
After {Q} until {R}	$::= \Box ((Q \wedge \neg R) \rightarrow ((\Diamond \llbracket \text{utb}(P) \rrbracket (P \vee R)) \mathcal{W} R))$

patterns, target logics, and third-party verification engines may not align. In addition, the specification of properties can be difficult for engineers who are not familiar with formal languages [4]. We need, therefore, some means to assist system engineers in this process. In particular, we wish to automate the process of weaving target logic mappings into the Structured English Grammar phrases. This is the domain of the *Property Specification Pattern Wizard* (PSPWizard), a tool for machine-assisted definition of temporal formulae capturing pattern-based system properties [18].

PSPFramework is designed to allow system engineers the flexibility to define system properties generically, without having to commit to writing a qualitative, real-time, or probabilistic specification in a specific logic formalism. This mechanism provides an effective means to map a given specification to various formats. PSPWizard, an implementation of PSPFramework, follows this approach. PSPWizard, once instantiated with a concrete target logic mapping, guides the system engineer through the property specification pattern catalog in a top-down fashion to indicate which patterns and subordinate attributes are supported in the selected target formalism.

A conceptual overview of the use of PSPWizard for property specification is shown in Fig. 3. For PSPWizard to work, it has to be instantiated with two components: an internalized representation of the Structured English Grammar (in

form of an abstract syntax) and a set of logic mappings that are arranged in tabular form, where rows represent a pattern instance and columns capture the mapping with respect to the selected scope.

These elements serve as a means to configure the GUI front-end of PSPWizard. Fig. 4 shows the tool in action. The user interface of PSPWizard is organized in six groups: scopes, patterns, Structured English Grammar, mapping, event/state definition, and PSPWizard options. By default, PSPWizard is enabled to support all pattern catalogued in PSPFramework, including all scope-specific variants. To instantiate a concrete pattern, one just has to select the scope (e.g., **Globally**), a fitting pattern category (e.g., **Order**), and the suitable pattern template within that category (e.g., *ResponseInvariance*) as illustrated in Fig. 4. The corresponding Structured English Grammar phrase and associated temporal logic formula (on the righthand side of the user interface) are updated instantaneously every time the selected pattern is changed. This is PSPWizard's way of providing instant feedback and assisting the system engineer in the effective formulation of desired system properties. A selected temporal logic may not support a given feature or may not support a pattern at all. PSPWizard renders such instances void. For example, the property captured in Fig. 4 entails a probability bound and MTL is presently selected as the target logic. However, MTL does not support

TABLE 4
Order Patterns (Precedence Family and Until) Mapping to Metric Temporal Logic

Precedence:	
Globally	$::= \Box (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow \Diamond \llbracket \text{gap}(P) \rrbracket S)$
Before $\{R\}$	$::= \Diamond R \rightarrow (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P) \rrbracket S \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R)) \mathcal{U} R$
After $\{Q\}$	$::= \Box (Q \rightarrow \Box (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow \Diamond \llbracket \text{gap}(P) \rrbracket S))$
Between $\{Q\}$ and $\{R\}$	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P) \rrbracket S \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R)) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::= \Box ((Q \wedge \neg R) \rightarrow (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P) \rrbracket S \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R)) \mathcal{W} R)$
PrecedenceChain_{1N}:	
Globally	$::= \Box (\Diamond \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket) \rightarrow \Diamond \llbracket \text{maxgap}(S) \rrbracket P)$
Before $\{R\}$	$::= \Diamond R \rightarrow (\Diamond \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket) \rightarrow (\Diamond \llbracket \text{maxgap}(S) \rrbracket P \vee \Diamond \llbracket \text{gap}(N-1, S) \rrbracket R) \mathcal{U} R)$
After $\{Q\}$	$::= \Box (Q \rightarrow \Box (P \rightarrow (\Diamond \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket))))$
Between $\{Q\}$ and $\{R\}$	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((\Diamond \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket)) \rightarrow (\Diamond \llbracket \text{maxgap}(S) \rrbracket P \vee \Diamond \llbracket \text{gap}(N-1, S) \rrbracket R)) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::= \Box ((Q \wedge \neg R) \rightarrow ((\Diamond \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket)) \rightarrow (\Diamond \llbracket \text{maxgap}(S) \rrbracket P \vee \Diamond \llbracket \text{gap}(N-1, S) \rrbracket R)) \mathcal{W} R)$ with $\llbracket Ch(i) \rrbracket = \wedge \bigcirc (\Diamond \llbracket \text{utb}(T_i) \rrbracket (T_i \llbracket Ch(i+1) \rrbracket))$
ConstrainedPrecedenceChain_{1N}:	
Globally	$::= \Box (\llbracket \text{cnt}(S) \rrbracket \mathcal{U} \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket) \rightarrow \Diamond \llbracket \text{maxgap}(S) \rrbracket P)$
Before $\{R\}$	$::= \Diamond R \rightarrow (\llbracket \text{cnt}(S) \rrbracket \mathcal{U} \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket) \rightarrow (\Diamond \llbracket \text{maxgap}(S) \rrbracket P \vee \Diamond \llbracket \text{gap}(N-1, S) \rrbracket R) \mathcal{U} R)$
After $\{Q\}$	$::= \Box (Q \rightarrow \Box (P \rightarrow (\llbracket \text{cnt}(S) \rrbracket \mathcal{U} \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket))))$
Between $\{Q\}$ and $\{R\}$	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((\llbracket \text{cnt}(S) \rrbracket \mathcal{U} \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket)) \rightarrow (\Diamond \llbracket \text{maxgap}(S) \rrbracket P \vee \Diamond \llbracket \text{gap}(N-1, S) \rrbracket R)) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::= \Box ((Q \wedge \neg R) \rightarrow ((\llbracket \text{cnt}(S) \rrbracket \mathcal{U} \llbracket \text{trigger}(S) \rrbracket (S \llbracket Ch(1) \rrbracket)) \rightarrow (\Diamond \llbracket \text{maxgap}(S) \rrbracket P \vee \Diamond \llbracket \text{gap}(N-1, S) \rrbracket R)) \mathcal{W} R)$ with $\llbracket Ch(i) \rrbracket = \wedge \llbracket \text{cnt}(T_i) \rrbracket \wedge \bigcirc (\llbracket \text{cnt}(T_i) \rrbracket \mathcal{U} \llbracket \text{utb}(T_i) \rrbracket (T_i \llbracket Ch(i+1) \rrbracket))$
PrecedenceChain_{N1}:	
Globally	$::= \Box (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket)))$
Before $\{R\}$	$::= \Diamond R \rightarrow ((\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))) \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R) \mathcal{U} R)$
After $\{Q\}$	$::= \Box (Q \rightarrow \Box (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))))$
Between $\{Q\}$ and $\{R\}$	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))) \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::= \Box ((Q \wedge \neg R) \rightarrow ((\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow (\Diamond \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))) \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R) \mathcal{W} R)$ with $\llbracket Ch(i) \rrbracket = \wedge \bigcirc (\Diamond \llbracket \text{gap}(P, i) \rrbracket (T_i \llbracket Ch(i+1) \rrbracket))$
ConstrainedPrecedenceChain_{N1}:	
Globally	$::= \Box (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow ((\neg P \wedge \llbracket \text{cnt}(P) \rrbracket) \mathcal{U} \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket)))$
Before $\{R\}$	$::= \Diamond R \rightarrow (((\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow ((\neg P \wedge \llbracket \text{cnt}(P) \rrbracket) \mathcal{U} \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))) \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R) \mathcal{U} R)$
After $\{Q\}$	$::= \Box (Q \rightarrow \Box (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow ((\neg P \wedge \llbracket \text{cnt}(P) \rrbracket) \mathcal{U} \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))))$
Between $\{Q\}$ and $\{R\}$	$::= \Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow ((\neg P \wedge \llbracket \text{cnt}(P) \rrbracket) \mathcal{U} \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))) \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::= \Box ((Q \wedge \neg R) \rightarrow (\Diamond \llbracket \text{trigger}(P) \rrbracket P \rightarrow ((\neg P \wedge \llbracket \text{cnt}(P) \rrbracket) \mathcal{U} \llbracket \text{gap}(P, 0) \rrbracket (S \llbracket Ch(1) \rrbracket))) \vee \Diamond \llbracket \text{elapsed}(P) \rrbracket R) \mathcal{W} R)$ with $\llbracket Ch(i) \rrbracket = \wedge \neg P \wedge \llbracket \text{cnt}(T_i) \rrbracket \wedge \bigcirc ((\neg P \wedge \llbracket \text{cnt}(T_i) \rrbracket) \mathcal{U} \llbracket \text{gap}(P, i) \rrbracket (T_i \llbracket Ch(i+1) \rrbracket))$
Until:	
Globally	$::= P \mathcal{U} \llbracket \text{time}(P) \rrbracket S$
Before $\{R\}$	$::= \Diamond \llbracket \text{mintime}(P) \rrbracket R \rightarrow ((P \wedge \neg R) \mathcal{U} \llbracket \text{time}(P) \rrbracket (S \vee R))$
After $\{Q\}$	$::= \Box (Q \rightarrow (P \mathcal{U} \llbracket \text{time}(P) \rrbracket S))$
Between $\{Q\}$ and $\{R\}$	$::= \Box ((Q \wedge \neg R \wedge \Diamond \llbracket \text{mintime}(P) \rrbracket R) \rightarrow ((P \wedge \neg R) \mathcal{U} \llbracket \text{time}(P) \rrbracket (S \vee R)))$
After $\{Q\}$ until $\{R\}$	$::= \Box ((Q \wedge \neg R) \rightarrow ((P \mathcal{U} \llbracket \text{time}(P) \rrbracket S) \mathcal{W} R))$

probabilities, Hence, the MTL formula is restricted to the qualitative and real-time aspects of the current property.

PSPWizard does not define any atomic propositions *per se*. However, the event interface allows the user to populate the system with a set of standard events (called sample events like P, Q, R, S, T, etc.), define new events (or states), and update the specification of an event, if required. In addition, one can choose between three forms of representing events in patterns: event name, event specification, or a combination thereof. The use of event name is recommended for generating compact target logic mappings. Event specification allows for the most precise characterization of the Structured English Grammar phrase. Using both allows for an explicit representation of the relationship between name and meaning of an event in property specification.

To illustrate the last aspect further, consider the property being captured in PSPWizard shown in Fig. 5. It reads:

Globally,
if $\{ \text{the MANET Synchronization component sends its time reference to the OMAP Synchronization component} \}$
then in response $\{ \text{the OMAP Synchronization component must forward this message to OMAP} \}$ **within** 100 ms.

which PSPWizard translates, using the embedded MTL mapping, into

$\Box ((\text{the MANET Synchronization component sends its time reference to the OMAP Synchronization component}) \rightarrow \Diamond [0, 100] (\text{the OMAP Synchronization component must forward this message to OMAP})).$

PSPWizard demonstrates the feasibility of a machine-assisted selection and parametrization of temporal logical properties in terms of specification patterns. The front-end of PSPWizard emulates the necessary design choices made by a system engineer to capture system properties. Whether or not a specific attribute can be expressed in the target logic is made explicit in the user interface of PSPWizard. Moreover, the automated translations of property specifications to temporal logic formulae enables a system engineer to focus on the system attribute rather than how that system attribute must be encoded in the target logic.

PSPWizard is a mature tool and has undergone several refinements since its inception. It is available for download,³ both as source code and as binary. It runs on all platforms supporting Java 7. Supported target logic mappings include LTL and MTL as well as an experimental mapping to the Prism property specification language [43].

3. Available online at: <http://ict.swin.edu.au/personal/mlumpe/PSPWizard>

TABLE 5
Order Patterns (Response Family) Mapping to Metric Temporal Logic

Response:		
Globally	$::=$	$\Box (P \rightarrow \Diamond^{\llbracket \text{time}(S) \rrbracket} S)$
Before $\{R\}$	$::=$	$\Diamond^{\llbracket \text{ltime}(S) \rrbracket} R \rightarrow (P \rightarrow (\neg R \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{U} R$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (P \rightarrow \Diamond^{\llbracket \text{time}(S) \rrbracket} S))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \Box^{\llbracket \text{umtime}(S) \rrbracket} \neg R \wedge \Diamond^{\llbracket \text{ltime}(S) \rrbracket} R) \rightarrow (P \rightarrow (\neg R \wedge \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R)))) \mathcal{U} R$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box ((Q \wedge \Box^{\llbracket \text{umtime}(S) \rrbracket} \neg R) \rightarrow ((P \rightarrow (\neg R \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{W} R))$
ConstrainedResponse:		
Globally	$::=$	$\Box (P \rightarrow \llbracket \text{cnt}(S) \rrbracket \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} S)$
Before $\{R\}$	$::=$	$\Diamond^{\llbracket \text{ltime}(S) \rrbracket} R \rightarrow (P \rightarrow ((\neg R \wedge \llbracket \text{cnt}(S) \rrbracket) \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{U} R$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (P \rightarrow \llbracket \text{cnt}(S) \rrbracket \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} S))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \Box^{\llbracket \text{umtime}(S) \rrbracket} \neg R \wedge \Diamond^{\llbracket \text{ltime}(S) \rrbracket} R) \rightarrow (P \rightarrow ((\neg R \wedge \llbracket \text{cnt}(S) \rrbracket) \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box ((Q \wedge \Box^{\llbracket \text{umtime}(S) \rrbracket} \neg R) \rightarrow ((P \rightarrow ((\neg R \wedge \llbracket \text{cnt}(S) \rrbracket) \mathcal{U}^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{W} R))$
ResponseChain_{1N}:		
Globally	$::=$	$\Box (P \rightarrow (\Diamond^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}(1) \rrbracket)))$
Before $\{R\}$	$::=$	$\Diamond R \rightarrow (P \rightarrow (\neg R \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}_R(1) \rrbracket))) \mathcal{U} R$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (P \rightarrow (\Diamond^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}(1) \rrbracket)))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \Diamond R) \rightarrow (P \rightarrow (\neg R \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}_R(1) \rrbracket))) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box (Q \rightarrow (P \rightarrow (\neg R \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}_R(1) \rrbracket))) \mathcal{W} R)$
with $\llbracket \text{Ch}(i) \rrbracket = \wedge \Diamond (\Diamond^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}(i+1) \rrbracket))$, and $\llbracket \text{Ch}_R(i) \rrbracket = \wedge \neg R \wedge \Diamond (\Diamond^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}_R(i+1) \rrbracket))$		
ConstrainedResponseChain_{1N}:		
Globally	$::=$	$\Box (P \rightarrow (\llbracket \text{cnt}(S) \rrbracket \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}(1) \rrbracket)))$
Before $\{R\}$	$::=$	$\Diamond R \rightarrow (P \rightarrow ((\neg R \wedge \llbracket \text{cnt}(S) \rrbracket) \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}_R(1) \rrbracket))) \mathcal{U} R$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (P \rightarrow (\llbracket \text{cnt}(S) \rrbracket \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}(1) \rrbracket)))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \Diamond R) \rightarrow (P \rightarrow ((\neg R \wedge \llbracket \text{cnt}(S) \rrbracket) \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}_R(1) \rrbracket))) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box (Q \rightarrow (P \rightarrow ((\neg R \wedge \llbracket \text{cnt}(S) \rrbracket) \mathcal{U}^{\llbracket \text{umtime}(S) \rrbracket} (S \llbracket \text{Ch}_R(1) \rrbracket))) \mathcal{W} R)$
with $\llbracket \text{Ch}(i) \rrbracket = \wedge \llbracket \text{cnt}(T_i) \rrbracket \wedge \Diamond (\llbracket \text{cnt}(T_i) \rrbracket \mathcal{U}^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}(i+1) \rrbracket))$, and $\llbracket \text{Ch}_R(i) \rrbracket = \wedge \neg R \wedge \llbracket \text{cnt}(T_i) \rrbracket \wedge \Diamond (\llbracket \text{cnt}(T_i) \rrbracket \mathcal{U}^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}_R(i+1) \rrbracket))$		
ResponseChain_{N1}:		
Globally	$::=$	$\Box (S \llbracket \text{Ch}(1) \rrbracket)$
Before $\{R\}$	$::=$	$\Diamond R \rightarrow ((S \llbracket \text{Ch}_R(1) \rrbracket) \mathcal{U} R)$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (S \llbracket \text{Ch}(1) \rrbracket))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((S \llbracket \text{Ch}_R(1) \rrbracket) \mathcal{U} R))$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box ((Q \wedge \neg R) \rightarrow ((S \llbracket \text{Ch}_R(1) \rrbracket) \mathcal{W} R))$
with $\llbracket \text{Ch}(i < N) \rrbracket = \wedge \Diamond (\Diamond^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}(i+1) \rrbracket))$ terminating in $\llbracket \text{Ch}(N) \rrbracket = \rightarrow \Diamond^{\llbracket \text{utb}(P) \rrbracket} P$, and $\llbracket \text{Ch}_R(i < N) \rrbracket = \wedge \Diamond (\neg R \mathcal{U}^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}_R(i+1) \rrbracket))$ terminating in $\llbracket \text{Ch}_R(N) \rrbracket = \rightarrow \Diamond^{\llbracket \text{utb}(P) \rrbracket} P$		
ConstrainedResponseChain_{N1}:		
Globally	$::=$	$\Box (S \llbracket \text{Ch}(1) \rrbracket)$
Before $\{R\}$	$::=$	$\Diamond R \rightarrow ((S \llbracket \text{Ch}_R(1) \rrbracket) \mathcal{U} R)$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (S \llbracket \text{Ch}(1) \rrbracket))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \neg R \wedge \Diamond R) \rightarrow ((S \llbracket \text{Ch}_R(1) \rrbracket) \mathcal{U} R))$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box ((Q \wedge \neg R) \rightarrow ((S \llbracket \text{Ch}_R(1) \rrbracket) \mathcal{W} R))$
with $\llbracket \text{Ch}(i < N) \rrbracket = \wedge \Diamond (\llbracket \text{cnt}(T_i) \rrbracket \mathcal{U}^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}(i+1) \rrbracket))$ terminating in $\llbracket \text{Ch}(N) \rrbracket = \rightarrow \llbracket \text{cnt}(P) \rrbracket \mathcal{U}^{\llbracket \text{utb}(P) \rrbracket} P$, and $\llbracket \text{Ch}_R(i < N) \rrbracket = \wedge \Diamond (\neg R \wedge \llbracket \text{cnt}(T_i) \rrbracket \mathcal{U}^{\llbracket \text{utb}(T_i) \rrbracket} (T_i \llbracket \text{Ch}_R(i+1) \rrbracket))$ terminating in $\llbracket \text{Ch}_R(N) \rrbracket = \rightarrow \llbracket \text{cnt}(P) \rrbracket \mathcal{U}^{\llbracket \text{utb}(P) \rrbracket} P$		
ResponseInvariance:		
Globally	$::=$	$\Box (P \rightarrow \Box^{\llbracket \text{time}(S) \rrbracket} S)$
Before $\{R\}$	$::=$	$\Diamond^{\llbracket \text{ltime}(S) \rrbracket} R \rightarrow ((P \rightarrow (\Box^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{U} R)$
After $\{Q\}$	$::=$	$\Box (Q \rightarrow \Box (P \rightarrow \Box^{\llbracket \text{time}(S) \rrbracket} S))$
Between $\{Q\}$ and $\{R\}$	$::=$	$\Box ((Q \wedge \Box^{\llbracket \text{umtime}(S) \rrbracket} \neg R \wedge \Diamond^{\llbracket \text{ltime}(S) \rrbracket} R) \rightarrow (P \rightarrow (\Box^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R))) \mathcal{U} R)$
After $\{Q\}$ until $\{R\}$	$::=$	$\Box ((Q \wedge \Box^{\llbracket \text{umtime}(S) \rrbracket} \neg R) \rightarrow ((P \rightarrow \Box^{\llbracket \text{time}(S) \rrbracket} (S \wedge \neg R)) \mathcal{W} R))$

TABLE 6
PSPFramework Mapping Support

$\llbracket \text{cnt}(P) \rrbracket$	$=$	$\neg Z^P$, if $\text{Constraint}(P)$ is defined, otherwise $true$
$\llbracket \text{time}(P) \rrbracket$	$=$	$[\llbracket t_l^P \rrbracket, \llbracket t_u^P \rrbracket]$, if $\llbracket t_u^P \rrbracket \neq \infty$, otherwise $[\llbracket t_l^P \rrbracket, \infty)$
$\llbracket \text{ltime}(P) \rrbracket$	$=$	$[\llbracket t_l^P \rrbracket, \infty)$
$\llbracket \text{umtime}(P) \rrbracket$	$=$	$[0, \llbracket t_l^P \rrbracket]$
$\llbracket \text{utb}(P) \rrbracket$	$=$	$[0, t_u^P]$
$\llbracket \text{trigger}(P) \rrbracket$	$=$	$[t_u^P, t_u^P]$
$\llbracket \text{gap}(P) \rrbracket$	$=$	$[0, t_u^P - t_l^P]$
$\llbracket \text{elapsed}(P) \rrbracket$	$=$	$[0, t_u^P]$
$\llbracket \text{maxgap}(P) \rrbracket$	$=$	$[0, t_u^P - t_l^P]$
$\llbracket \text{gap}(n, P) \rrbracket$	$=$	$[0, t_u^P + \sum_{i=1}^n t_u^{T_i}]$
$\llbracket \text{gap}(P, n) \rrbracket$	$=$	$[0, t_u^P - t_l^P - \sum_{i=1}^n t_u^{T_i}]$
$\llbracket t_l^P \rrbracket$	$=$	t_l^P , if t_l^P is defined, otherwise 0
$\llbracket t_u^P \rrbracket$	$=$	t_u^P , if t_u^P is defined, otherwise ∞
$\llbracket \bowtie p \rrbracket$	$=$	$\begin{cases} < p, & \text{if Probability is defined using the 'lower than' option} \\ \leq p, & \text{if Probability is defined using the 'lower than or equal' option} \\ > p, & \text{if Probability is defined using the 'greater than' option} \\ \geq p, & \text{if Probability is defined using the 'greater than or equal' option} \end{cases}$

TABLE 7
Universality Property Pattern Mapped to LTL, CTL, TCTL, CSL, and PLTL

LTL	Globally	$::= \Box(P)$
	Before $\{R\}$	$::= \Diamond R \rightarrow (P \mathcal{U} R)$
	After $\{Q\}$	$::= \Box(Q \rightarrow \Box(P))$
	Between $\{Q\}$ and $\{R\}$	$::= \Box((Q \wedge \neg R \wedge \Diamond R) \rightarrow (P \mathcal{U} R))$
	After $\{Q\}$ until $\{R\}$	$::= \Box(Q \wedge \neg R \rightarrow (P \mathcal{W} R))$
CTL	Globally	$::= AG(P)$
	Before $\{R\}$	$::= A[(P \vee AG(\neg R)) \mathcal{W} R]$
	After $\{Q\}$	$::= AG(Q \rightarrow AG(P))$
	Between $\{Q\}$ and $\{R\}$	$::= AG(Q \wedge \neg R \rightarrow A[(P \vee AG(\neg R)) \mathcal{W} R])$
	After $\{Q\}$ until $\{R\}$	$::= AG(Q \wedge \neg R \rightarrow A[P \mathcal{W} R])$
TCTL	Globally	$::= AG^{\llbracket \text{time}(P) \rrbracket} (P)$
	Before $\{R\}$	$::= A(((P \vee AG(\neg R)) \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R) \vee (G^{\llbracket \text{time}(P) \rrbracket} (P \wedge \neg R) \mathcal{U}^{\llbracket \text{maxtime}(P) \rrbracket} R))$
	After $\{Q\}$	$::= AG(Q \rightarrow AG^{\llbracket \text{time}(P) \rrbracket} (P))$
	Between $\{Q\}$ and $\{R\}$	$::= AG((Q \wedge AG^{\llbracket \text{untime}(P) \rrbracket} (\neg R)) \rightarrow A(((P \vee AG(\neg R)) \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R) \vee (G^{\llbracket \text{time}(P) \rrbracket} (P \wedge \neg R) \mathcal{U}^{\llbracket \text{maxtime}(P) \rrbracket} R)))$
	After $\{Q\}$ until $\{R\}$	$::= AG((Q \wedge AG^{\llbracket \text{untime}(P) \rrbracket} (\neg R)) \rightarrow A(P \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} R))$
CSL	Globally	$::= \mathcal{P}_{\leq \Delta P}(\Box^{\llbracket \text{time}(P) \rrbracket} (P))$
	Before $\{R\}$	$::= \mathcal{P}_{\leq \Delta P}(\Diamond^{\llbracket \text{lmin}(P) \rrbracket} R \rightarrow ((P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R) \vee \mathcal{P}_{=1}(\Box^{\llbracket \text{time}(P) \rrbracket} (P))))$
	After $\{Q\}$	$::= \mathcal{P}_{=1}(\Box(Q \rightarrow \mathcal{P}_{\leq \Delta P}(\Box^{\llbracket \text{time}(P) \rrbracket} (P))))$
	Between $\{Q\}$ and $\{R\}$	$::= \mathcal{P}_{=1}(\Box((Q \wedge \mathcal{P}_{=1}(\Box^{\llbracket \text{untime}(P) \rrbracket} (\neg R)) \rightarrow \mathcal{P}_{\leq \Delta P}(\Diamond^{\llbracket \text{lmin}(P) \rrbracket} R \rightarrow ((P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R) \vee \mathcal{P}_{=1}(\Box^{\llbracket \text{time}(P) \rrbracket} (P))))))$
	After $\{Q\}$ until $\{R\}$	$::= \mathcal{P}_{=1}(\Box((Q \wedge \mathcal{P}_{=1}(\Box^{\llbracket \text{untime}(P) \rrbracket} (\neg R)) \rightarrow \mathcal{P}_{\leq \Delta P}(P \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} R)))$
PLTL	Globally	$::= \Box^{\llbracket \text{time}(P) \rrbracket} (P) \llbracket \Delta P \rrbracket$
	Before $\{R\}$	$::= \Diamond^{\llbracket \text{lmin}(P) \rrbracket} R \rightarrow ((P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R) \vee (\Box^{\llbracket \text{time}(P) \rrbracket} (P))) \llbracket \Delta P \rrbracket$
	After $\{Q\}$	$::= \Box(Q \rightarrow \Box^{\llbracket \text{time}(P) \rrbracket} (P)) \llbracket \Delta P \rrbracket$
	Between $\{Q\}$ and $\{R\}$	$::= \Box((Q \wedge \Box^{\llbracket \text{untime}(P) \rrbracket} (\neg R) \wedge \Diamond^{\llbracket \text{lmin}(P) \rrbracket} R) \rightarrow \Diamond^{\llbracket \text{lmin}(P) \rrbracket} R \rightarrow ((P \mathcal{U}^{\llbracket \text{time}(P) \rrbracket} R) \vee (\Box^{\llbracket \text{time}(P) \rrbracket} (P)))) \llbracket \Delta P \rrbracket$
	After $\{Q\}$ until $\{R\}$	$::= \Box((Q \wedge \Box^{\llbracket \text{untime}(P) \rrbracket} (\neg R)) \rightarrow (P \mathcal{W}^{\llbracket \text{time}(P) \rrbracket} R)) \llbracket \Delta P \rrbracket$

Nevertheless, a direct application of specification patterns, as they exist in a catalogue, may not be sufficient for specifying particular properties. For instance, Dwyer et al. [4] observed that some properties require a variant of a pattern in which one can specify a left-open or right-closed

scope rather than the more common left-closed and right-open versions used in a catalogue. The current version of PSPWizard does not offer this kind of flexibility. This is one of the main arguments for a continuing improvement and refinement of the capabilities of PSPWizard.

Moreover, certain requirements to be rendered to temporal logic formulae might be incomplete, inconsistent, or described at a level of abstraction that hampers a faithful description of the reality [10]. A further exploration and analysis of the requirements is required in this case. The current version of PSPWizard and the underlying Structured English Grammar cannot be used for this purpose. For example, the PROPEL system [22], [33] guides the user to explicitly capture subtle, but important, details that need to be considered while formally specifying requirements (cf. Section 2.4). Enhancements to PSPWizard in this direction are possible by adopting the features offered in PROPEL.

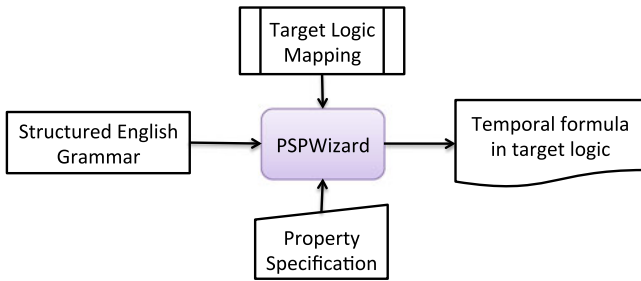


Fig. 3. Property specification in PSPWizard.

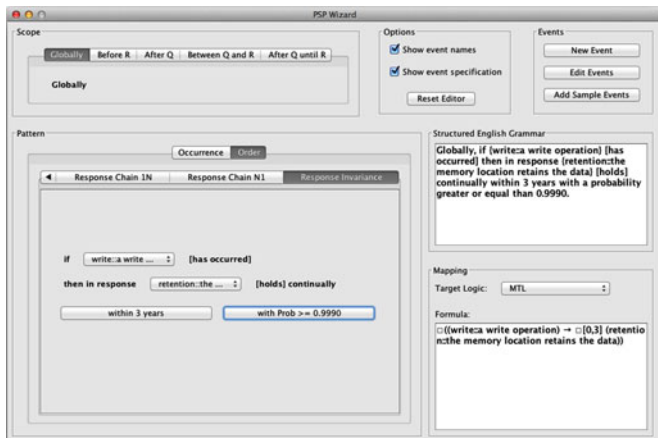


Fig. 4. PSPWizard in action.

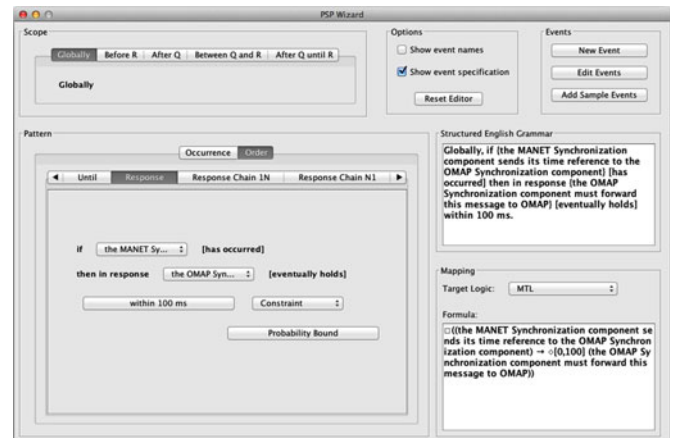


Fig. 5. Faithful property mapping in PSPWizard.

TABLE 8
Examples of the New Occurrence Patterns

Occurrence	Qualitative Patterns	Real-time Patterns	Probabilistic Patterns
Absence	[4]	After an erroneous authentication, a new authentication cannot be performed within 5 seconds. (<i>scope: after Q; source: E-commerce application</i>)	The probability of a duplicate record for the same vehicle at a toll gate gantry occurring within one year is less than 1%. (<i>scope: global; source: electronic billing system for toll roads</i>)
Universality	[4]	After the satellite is position in stable orbit, all control parameters need to be consistent between the primary and backup system for the 3 year mission time. (<i>scope: after Q; source: satellite control system</i>)	Data in non-volatile memory must persist with a probability of 99.999% for a duration of 3 years. (<i>scope: global; source: satellite control system</i>)
Existence	[4]	Between an <code>addToCart()</code> operation and an <code>EmptyCart(true)</code> operation there must be a <code>removeItem(d1)</code> or <code>makeAnOrder()</code> operation within the session timeout (5 minutes). (<i>scope: between Q and R; source: E-commerce application</i>).	[6]
Bounded Existence	[4]	N/A	N/A
Minimum Duration	N/A	[5]	Once the water pump is turned off, it remains so for at least 30 seconds with the probability greater or equal than 99%. (<i>scope: global; source: insulin pump [42]</i>)
Maximum Duration	N/A	[5]	Once the pump engine of an insulin pump control system starts working, it remains so for at most a period that is time defined by the delivery time of the control system according to the patient's glucose value, with a probability greater or equal than 99.999%. (The engine working time defines the quantity of insulin to be injected) (<i>scope: global; source: insulin pump [42]</i>)
Recurrence	The control system periodically checks the availability of its sensors. (<i>scope: global; source: satellite control system</i>)	[5]	A system backup is performed repeatedly every night with a probability greater or equal than 99.99%. (<i>scope: global; source: standard requirements for business information system</i>)

5 VALIDATION

This research makes two contributions. First, we have identified a new set of specification patterns to fill identified gaps in the current theory of property specification patterns. Second, we have created a novel framework, including associated tool support, to facilitate property specification through the application of property specification patterns.

In this section, we discuss the appropriateness and accuracy of our approach. In particular, we show, in Section 5.1, that the newly discovered specification patterns address actual industrial specification problems, and explore, in Section 5.2, a concrete industrial case study arising from the European Presto project⁴ to illustrate how PSPFramework adds specific value to the process of capturing industrial requirements.

5.1 Patterns Validation

In Section 3, we demonstrated how to infer new patterns from an analysis of existing qualitative, real-time, and probabilistic property specification patterns. However, as stated by Alexander et al. [12]:

“Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

Consequently, to irrefutably attest the applicability of the newly discovered patterns, the problems associated with those patterns must occur in real-world settings. We mined, therefore, actual industrial requirements for evidence of the presents or applicability of the newly discovered patterns. Our findings are summarized in Tables 8 and 9. For those patterns that had been previously validated, corresponding literature references are provided in the cells. For new

patterns, we describe an industrial application in the corresponding cell. The patterns *Transient State Probability*, *Steady State Probability*, *Precedence*, and *Response* are omitted from Tables 8 and 9, as they have already been extensively studied in previous work [2], [5], [6].

5.2 Validation of the Framework through an Industrial Case Study

In order to assess the effectiveness of PSPFramework, we applied it to an industrial case study. This case study enables us to demonstrate how our framework facilitates requirements formalization in a concise and reliable way. The case study concerns the development of an indoor positioning system comprising a series of interacting *Mobile Ad-Hoc Network* devices (DM) and *Frequency Hopping Ultra Wide-Band* devices (DU). This project has been proposed by THALES Italy in the context of the European Presto project (see <http://www.presto-embedded.eu/>).

Mobile Ad-hoc Networks (MANET) are based on the *Optimized Link State Routing* (OLSR) (<http://www.olsr.org/>) and IEEE 802.15.4 (<http://ieee802.org/>) protocols. OLSR, documented in the experimental Request For Comment (RFC) 3626, is a table-driven and pro-active IP routing protocol optimized for MANET. OLSR's core functionality distinguishes three main modules: *Neighbor Sensing*, *Multipoint Relaying* (MPR) and *Link-State Flooding*. Most control traffic in a MANET arises from dynamically updating repositories maintained by OLSR in response to receiving control messages (cf. Fig. 7).

An implementation of the OLSR and IEEE 802.15.4 protocols involve real-time message processing, message generation, and route calculation. Control and data messages are required to be relayed by different components in the model. We used PSPFramework and the applicable specification patterns to capture the properties of the model. Table 10 summarizes the list of requirements and their formalization in Structured English Grammar.

This industrial case study reveals two crucial aspects with respect to the validation of the framework. The

4. PRESTO project (ARTEMIS-2010-1-269362), co-funded by the European Commission under the ARTEMIS Joint Undertaking (JU) initiative: <http://www.presto-embedded.eu/>.

TABLE 9
Examples of the New Order Patterns

Order	Qualitative Patterns	Real-time Patterns	Probabilistic Patterns
Response Chain: 2 stimuli / 1 response	[4]	If the optical smoke detector and the ionization detector raises a smoke alarm then an alarm sound alerts occupants within 2s (<i>scope: global; source: house smoke detector</i>)	If a car is detected at a gantry and the speed measurement indicates that the speed of the car is greater than 130km/h then an image made by the camera is sent to the operator within 1 second in 99.999 percent of the cases (<i>scope: global; source: automatic toll road billing system</i>)
Precedence Chain: 2 causes / 1 effect	[4]	cf. Section III, ABS	The brake assist system will only be activated if the electronic control unit detects (i) a wheel rotating significantly slower than the others and (ii) the valves have been actuated to reduce hydraulic pressure to the brake at the affected wheel with a certainty of 99.9%. (<i>scope: global; source: anti-lock brake system – http://en.wikipedia.org/wiki/Anti-lock_braking_system</i>)
Constrained Response	When a credit card fraudulent transaction is detected or reported the case will be investigated and till the investigation is finalized no further account transactions must occur. (<i>scope: global; source: banking regulations</i>)	Once a shutdown command has occurred, the system will successfully recover within 2 minutes and without any further failures occurring. (<i>scope: global; source: satellite control system</i>)	[6]
Constrained Response Chain: 1 stimulus / 2 responses	[4]	If a cargo controller signals a smoke warning then the air ventilation system is turned off and then the compartment isolation valves are closed within 60 seconds unless the system or the operator recognized a false warning. (<i>scope: global; source: cargo smoke controller – http://www.tc.faa.gov/its/worldpac/techrpt/ar0758.pdf</i>)	If a road toll gantry fails to detect a RFID signal from a passing vehicle, then a camera takes a picture of the vehicle number plate 99.5% of the time and a record of an unpaid vehicle is created 99.99% of the time unless the roadside system is down (<i>source: roadside system in Australia</i>)
Constrained Precedence Chain: 2 causes / 1 effect	[4]	A Resolution Advisory (RA) is issued by ACAS if ACAS detects a potential collision with another aircraft followed by a Travel Advisory (TA) informing the pilot of the trajectory of the intruding aircraft, without the intruding aircraft changing course within 20 seconds. (<i>scope: global; source: http://www.icao.int/Meetings/anconf12/Document%20Archive/9863_cons_en.pdf</i>)	The driver is alerted via an audible signal if the speed of the car was greater than 65km/h, the distance between the car and the road lane markings decreased rapidly, and, in the meanwhile, the speed of the car was not decreased under 60km/h with a certainty of 99.9%. (<i>scope: global; source: Driver Alert Control (DAC) system of Volvo – https://www.media.volvocars.com/global/en-gb/media/pressreleases/12130</i>)
Response Invariance	After a write operation, the memory location should retain the data. (<i>scope: global; source: satellite control system</i>)	[5]	After a write operation, the memory location should retain the data for the next 3 years with a probability of 99.9%. (<i>scope: global; source: satellite control system</i>)
Until	Power to a system must be available until a shutdown command is issued. (<i>scope: global; source: satellite control system</i>)	After 30 ms of continuously operating the pump engine must be stopped. Note: the engine is connected to a piston rod which sends forward a plunger in order to deliver the insulin to the body. (<i>scope: global; source: insulin pump [42]</i>).	[6]

formalization of the requirements makes use of both the existing patterns and the newly identified patterns discovered in this work (new patterns are highlighted in light gray in Table 10 and by the keyword “(new)” under the pattern number). In addition, all requirements in this case study can be expressed by using solely the framework and its natural language interface.

The second aspect relates to the effectiveness of the Structured English Grammar. Table 10 presents the original requirements in the *Requirement* column and their formal representations in the *Structured English Grammar* column. Structured English Grammar provides an intuitive means for capturing requirements to system engineers. Structured English Grammar reduces the cognitive load and enables practitioners with no or limited expertise in formal methods and logics to collect and specify desired

system properties. Since the grammar is accompanied with mappings to suitable logic formalisms (cf. Section 4.2), given property specifications can be automatically translated into logic formulae. Hence, our approach provides a means for unambiguous representation of the requirements and enables automatic verification and reasoning. After using the PSPFramework, the engineers that were responsible for the requirements specification and formalization, positively commented on its usability and adoptability in the industrial context.

We argue, therefore, that a Structured English Grammar-based approach, like the one presented in this work, allows system engineers, who are unfamiliar with the use of

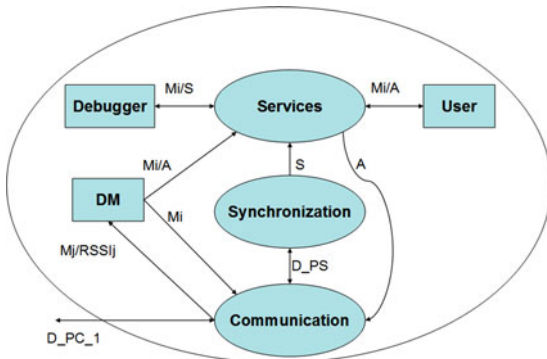


Fig. 6. DM Functionalities (taken from Deliverable D 2.1: Case studies modeling and design—use case specification [44]).

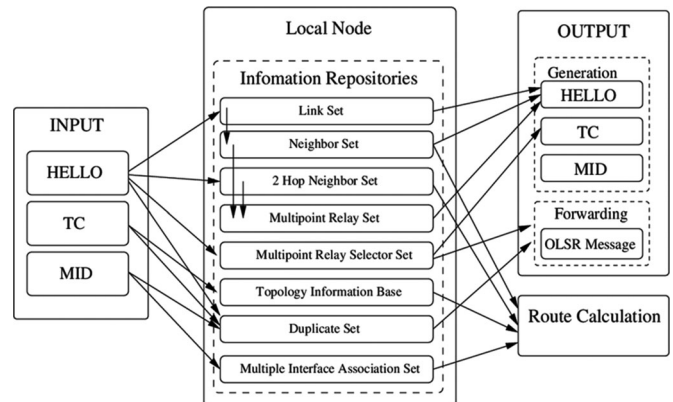


Fig. 7. OLSR information repositories relation overview (taken from Deliverable D 2.1: Case studies modeling and design—use case specification [44]).

TABLE 10
Thales Case Study: Requirements and Their Formalization

N.	Pattern	Requirement	Structured English Grammar
1	Response Property Pattern	The <i>MANET Synchronization</i> component will share its time reference with the <i>OMAP Synchronization</i> component that, in turn, will send this reference to <i>OMAP</i> with a maximum time of 100 microseconds.	Globally, if {the <i>MANET Synchronization</i> component sends its time reference to the <i>OMAP Synchronization</i> component} then in response {the <i>OMAP Synchronization</i> component must forward this message to <i>OMAP</i> } within 100ms.
2 (new)	Time-Constrained Absence	A DM cannot remain without a DM master associated after 2 seconds it is started.	Globally, it is never the case that {a DM remains without a DM master associated} after 2s.
3	Response Property Pattern	A generic DM_j that receives a synchronization message from a generic DM_i , will forward the synchronization to a generic DM_k . This property holds for every DM_i , DM_j , and DM_k .	Globally, if {a generic DM_i sends a synchronization message to a generic DM_j } then in response {this generic DM_j component must forward this message to a generic DM_k }.
4 (new)	Time-Constrained Universality	The time required to perform the network synchronization, i.e., the time needed for the network nodes to share a common temporal reference with the DM master, must be less than 7 seconds.	Globally, it is always the case that {the network synchronization is performed} within 7 seconds.
5	Response Property Pattern	An alarm is signaled by a user through its user interface and received by the Services component of DM_i . Once triggered, the alarm must be forwarded to the Communication component of DM_i . This property holds for every DM.	Globally, if {an alarm is signaled by a user through its user interface and received by the Services component,} then in response {the alarm is forwarded to the Communication component}.
6 (new)	Time-constrained Response Chain Property Pattern, 1 cause n effects, with $n \leq 49$	The alarm signal generated by a DM is received by the other DMs belonging to the network within 1 second.	Globally, if {the alarm signal has been generated by DM_1 } then in response {the alarm is received by DM_2 } followed by { DM_3 } ... followed by { DM_{49} } within 1 second. Note: This pattern should be instantiated for each DM_i with $1 \leq i \leq 49$
7	Response Property Pattern	Once a DM_i , DM_j , receives an alarm notification from another DM_k , the alarm must be notified to the user through the user interface. This property holds for every DM_i and DM_j .	Globally, if {an alarm is sent by a DM and received by another DM,} then in response {the alarm is notified to the user through its user interface}.
8 (new)	Time-constrained Response Chain Property Pattern, 1 cause n effects, with $n \leq 49$	The local instance of matrix M for DU_i must be updated with a frequency f so that any DU can update its navigation data every second. This implies that each DM transmits its local matrix instance to the other DM_s within $1/f$ seconds.	Globally, if {a clock has occurred,} then in response {the matrix update is sent from DM_1 to DM_2 } followed by { DM_3 } ... followed by { DM_{49} } within $1/f$ seconds. Note: This pattern should be instantiated for each DM_i with $1 \leq i \leq 49$
9	Response Property Pattern	Once DU_i sends the local matrix to the Services component, this component must send the display message to the User Interface. In this way the user is notified about the local matrix status. This property holds for every DM_i .	Globally, if { DU_i sends the local matrix to the Services component,} then in response {this component must send the display message to the User Interface}.
10 (new)	Time-constrained Response Chain Property Pattern, n causes 1 effect, with $n \leq 49$	The local instance of matrix M for DU_i must be updated with a frequency f so that any DU can update its navigation data every second. This implies that each DM transmits its local matrix instance to the other DM_s with a period of $1/f$ and that any DM is able to receive and elaborate up to 49 matrices in $1/f$ seconds.	Globally, if {a matrix update has been received by DM_1 from DM_2 } followed by { DM_3 } ... followed by { DM_{49} } then in response { DM_1 performs an update} within $1/f$ seconds. Note: This pattern should be instantiated for each DM_i with $1 \leq i \leq 49$
11	Response Chain Property Pattern, 1 cause 6 effects	As shown in Fig. 7, a generic DM_i sends a <i>HELLO</i> message to <i>OLSR</i> . Specifically, it is received from <i>Local Node</i> that interacts with the various <i>OLSR</i> information repositories. Then, <i>Local Node</i> must forward the <i>HELLO</i> message to <i>Link set</i> , and then to <i>Neighbor Set</i> that update their information. Finally, the <i>HELLO</i> message is sent to <i>MPR Set</i> , which activates a recalculation. This property holds for every DM_i .	Globally, if {an <i>HELLO</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to <i>Link Set</i> } followed by {the <i>Neighbor Set</i> component} followed by {an update performed by <i>Neighbor Set</i> }, followed by {a forwarding to <i>MPR Set</i> }, followed by {a recalculation performed by <i>MPR Set</i> }.
12	Response Chain Property Pattern, 1 cause 4 effects	As shown in Fig. 7, <i>Local Node</i> must also forward the <i>HELLO</i> message to the 2 Hop <i>Neighbor Set</i> component, which updates its information, and to the <i>MPR Set</i> component, thus triggering a recalculation. This property holds for every DM_i .	Globally, if {a <i>HELLO</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the 2 Hop <i>Neighbor Set</i> component} followed by {an update performed by the 2 Hop <i>Neighbor Set</i> component}, followed by {a forwarding to the <i>MPR Set</i> component}, followed by {a recalculation performed by the <i>MPR Set</i> component}.
13	Response Chain Property Pattern, 1 cause 2 effects	As shown in Fig. 7, the <i>Local Node</i> component has to forward the <i>HELLO</i> message also to the <i>MPR Selector Set</i> component which in turn is required to perform an update. This property holds for every DM_i .	Globally, if {a <i>HELLO</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the <i>MPR Selector Set</i> component} followed by {an update performed by this component}.
14	Response Chain Property Pattern, 1 cause 2 effects	As shown in Fig. 7, <i>Local Node</i> must forward the <i>HELLO</i> message to <i>Duplicate Set</i> , that in the case of the message is not yet performed, registers the message. To model that we used the alternative operator, whit an empty alternative. This property holds for every DM_i .	Globally, if {an <i>HELLO</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the <i>Duplicate Set</i> component} followed by {a registration performed by this component in the case of the message is not yet registered}.
15	Response Chain Property Pattern, 1 cause 2 effects	As shown in Fig. 7, <i>Local Node</i> must forward the <i>TC</i> message to <i>Duplicate Set</i> , that in the case of the message is not yet performed, registers the message. To model that we used the alternative operator, whit an empty alternative. This property holds for every DM_i .	Globally, if {an <i>TC</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the <i>Duplicate Set</i> component} followed by {a registration performed by this component in the case of the message is not yet registered}.
16	Response Chain Property Pattern, 1 cause 2 effects	As shown in Fig. 7, the <i>Local Node</i> component must forward the <i>MID</i> message to the <i>Duplicate Set</i> component, that in the case of the message is not yet performed, registers the message. For modelling that we used the alternative operator, whit an empty alternative. This property holds for every DM_i .	Globally, if {an <i>MID</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the <i>Duplicate Set</i> component} followed by {a registration performed by this component in the case of the message is not yet registered}.
17	Response Chain Property Pattern, 1 cause 2 effects	As shown in Fig. 7, in the case of <i>TC</i> messages sent from DM_i to <i>Local Node</i> , this last component is required to forward the message to the <i>Topology Information Base</i> component, which in turn must perform an update. This property holds for every DM_i .	Globally, if {a <i>TC</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the <i>Topology Information Base</i> component} followed by {an update performed by this component}.
18	Response Chain Property Pattern, 1 cause 2 effects	As shown in Fig. 7, in the case of <i>TC</i> messages sent from DM_i to the <i>Local Node</i> component, this last component is required to forward the message to the <i>Multiple Interface Association Set</i> component, which in turn must perform an update. This property holds for every DM_i .	Globally, if {a <i>MID</i> message has been sent from DM_i to the <i>Local Node</i> of the <i>OLSR</i> ,} then in response {the message is forwarded to the <i>Multiple Interface Association Set</i> component} followed by {an update performed by this component}.
19	Precedence Chain Property Pattern, 3 causes 1 effect	As shown in Fig. 7, <i>Local Node</i> before generating the message must query the <i>Link Set</i> , <i>Neighbor Set</i> and <i>MPR Set</i> components. We have a failure is an <i>HELLO</i> message is exchanged without performing a chain of operations: querying <i>LinkSet</i> , <i>Neighbor Set</i> and <i>MPR Set</i> . This property holds for every DM_i .	Globally, if {an <i>HELLO</i> message is sent from the <i>Local Node</i> of the <i>OLSR</i> to DM_i ,} then it must have been the case that { <i>Local Node</i> queried <i>Link Set</i> } and afterwards { <i>Local Node</i> queried <i>Neighbor Set</i> }, and afterwards { <i>Local Node</i> queried <i>MPR Set</i> } before {sending the <i>HELLO</i> message}.
20	Precedence Property Pattern	As shown in Fig. 7, analogously to what happen for <i>HELLO</i> messages, a <i>TC</i> message generation must be preceded by a query of <i>MPR Selector Set</i> . This property holds for every DM_i .	Globally, if {a <i>TC</i> message is sent from the <i>Local Node</i> of the <i>OLSR</i> to DM_i ,} then it must have been the case that { <i>Local Node</i> queried <i>MPR Selector Set</i> } before {sending the <i>TC</i> message}.
21	Precedence Chain Property Pattern, 2 causes 1 effect	As shown in Fig. 7, this property deals with the control traffic message that must be sent only after querying the <i>MPR Selector Set</i> and <i>Duplicate Set</i> components. This property holds for every DM_i .	Globally, if {a controlTraffic message is sent from the <i>Local Node</i> of the <i>OLSR</i> to DM_i ,} then it must have been the case that { <i>Local Node</i> queried <i>MPR Selector Set</i> } and afterwards { <i>Local Node</i> queried <i>Duplicate Set</i> } before {sending the controlTraffic message}.
22	Precedence Chain Property Pattern, 4 causes 1 effect	As shown in Fig. 7, the route calculation message must be sent only after querying the <i>Neighbor Set</i> 2 Hop, <i>Neighbor Set</i> , <i>Topology Information Base</i> , and <i>Multiple Interface Association Set</i> components. This property holds for every DM_i .	Globally, if {a routeCalculation message is sent from the <i>Local Node</i> of the <i>OLSR</i> to DM_i ,} then it must have been the case that { <i>Local Node</i> queried <i>Neighbor Set</i> } and afterwards { <i>Local Node</i> queried 2 Hop <i>Neighbor Set</i> } and afterwards { <i>Local Node</i> queried <i>Topology Information Base</i> } and afterwards { <i>Local Node</i> queried <i>Multiple Interface Association</i> } before {sending the routeCalculation message}.

temporal logic and formal verification, to produce correct formal specification of requirements.

6 CONCLUSIONS AND FUTURE WORK

"A pattern system does not belong to an individual, but to the community of experts and practitioners who contribute to and use it." (Dwyer et al. [20])

In this work, we have developed PSPFramework, a new and comprehensive pattern-based approach for the specification of qualitative, real-time, and probabilistic system attributes. PSPFramework encompasses three key ingredients: i) a unified property specification pattern catalogue, ii) a natural language interface in form of a Structured English Grammar, and iii) a set for mappings to suitable logic formalisms. PSPFramework is designed to effectively guide system engineers in their efforts to capture desired system properties concisely and with ease. To facilitate this process, PSPFramework is accompanied with PSPWizard, a tool that allows system engineers to disambiguate the potentially different interpretations of high-level requirements, and that assists in elucidating formal properties clearly by guiding system engineers through the different interpretation options.

Previous works on system property specification patterns in the area of qualitative [4], real-time [2], [5], and probabilistic [6] specification patterns have laid the foundations for our study. We have systematically catalogued existing specification patterns, and further extended an integrated catalogue to fill identified gaps. We found and extended 40 new specification patterns. The results of our work are captured in a unified property specification pattern catalogue, being documented also in a collaborative web-site [19]. As envisioned by Dwyer et al. [20], this unified catalogue can be shared and further enhanced by experts and practitioners as our understanding of patterns becomes more complete and refined over time.

Before this work, the focus on specification patterns was specialized. However, real-world property specification requires, in general, more than just one type of patterns. There was a need to bring the different approaches together in a single framework. We chose a top-down approach in analyzing and consolidating property specification patterns. Our method entailed a systematic analysis of the major groupings of properties and specification typing. This holistic view allowed us to set aside the silo approaches used thus far. In particular, we were able to identify previously unknown patterns that, both, have practical application and substantially improve our ability to capture system properties more effectively.

A pattern catalogue is work in progress. Our pattern catalogue is no exception. New patterns might be discovered while defining system properties for emerging domains. For instance, Haydar et al. [45] have developed a library of Web specification patterns, whereas the work by Bianculli et al. [14] has identified specification patterns that characterize service provisioning in service-based applications.

Although the patterns identified in our catalogue can be translated into at least one logic, disjointed specifications in different logic representations may create difficulties for system engineers. This calls for ensuring that different

interpretations of the same pattern in different logics are compatible. This issue presents a major challenge to effective verification of the real-world systems. The complexity and decidability of the verification problem may vary depending on the chosen semantics (e.g., dense- versus discrete-time semantics). For instance, the satisfiability of timing constraints expressing punctuality is known to be undecidable, and some approaches benefit from relaxing punctuality [46]. Furthermore, Cabrera-Castillos et al. [23] note that the semantics of some qualitative specification patterns are scope-dependent and varies with the underlying logic formalism. Cabrera-Castillos et al. [23] specifically compared Büchi automata and linear temporal logic. Similar to this work, a clear analysis for the semantics in different specification formalisms for real-time [2], [5], and probabilistic [6] specification patterns would be required. This will be an area of future work and needs to be addressed to effectively cope with the ever increasing complexity of requirements and properties in modern software systems.

A future direction of this work relates to the integration of PSPFramework and PSPWizard with existing model-based software engineering techniques and requirement engineering frameworks and tools. This integration would enable backward and forward traceability between system models and property specification patterns, between counter-examples and the invalid part of the pattern, etc. during verification. This requires coping with multiple artifacts and employing suitable techniques to keep these artifacts consistent by propagating relevance changes to them when necessary. Malavolta et al. [47] proposed an automated framework that allows architectural languages and tools interoperability through model transformation technologies which could serve as a basis in this direction.

ACKNOWLEDGMENTS

This work is partly supported by PRESTO project (ARTEMIS-2010-1-269362), co-funded by the European Commission under the ARTEMIS Joint Undertaking (JU) initiative: <http://www.presto-embedded.eu/>. This work is also partially supported by the DFG (German Research Foundation) under the Priority Programme SPP1593: Design For Future—Managed Software Evolution. The authors would also like to thank Indika Meedeniya for his contributions to and the realization of PSPWizard. Patrizio Pelliccione is the corresponding author of the article.

REFERENCES

- [1] M. Bozzano and A. Villafiorita, *Design and Safety Assessment of Critical Systems*. CRC Press, 2011.
- [2] P. Bellini, P. Nesi, and D. Rogai, "Expressing and organizing real-time specification patterns via temporal logics," *J. Syst. Softw.*, vol. 82, no. 2, pp. 183–196, Feb. 2009.
- [3] F. Kröger and S. Merz, *Temporal Logic and State Systems*. Springer, 2008.
- [4] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett, "Patterns in property specifications for finite-state verification," in *Proc. 21st Int. Conf. Softw. Eng.*, ACM, 1999, pp. 411–420.
- [5] S. Konrad and B. H. C. Cheng, "Real-time specification patterns," in *Proc. 27th Int. Conf. Softw. Eng.*, New York, NY, USA: ACM, 2005, pp. 372–381.
- [6] L. Grunske, "Specification patterns for probabilistic quality properties," in *Proc. 30th Int. Conf. Softw. Eng.*, New York, NY, USA: ACM, 2008, pp. 31–40.

- [7] E. Clarke, O. Grumberg, and D. Peled, *Model Checking*, MIT Press, 2000.
- [8] D. Peled, P. Pelliccione, and P. Spoletini, *Wiley Encyclopedia of Computer Science and Engineering 6th Edition, 5-Volume Set.*, Published by John Wiley & Sons in Hoboken, NJ, 2009, vol. 3, ch. Model Checking, pp. 1904–1920.
- [9] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers—Principles, Techniques, & Tools*, 2nd ed. Pearson Education Inc., 2007.
- [10] T. H. Nguyen, B. Q. Vo, M. Lumpe, and J. Grundy, “KBRE: A framework for knowledge-based requirements engineering,” *Softw. Q. J.*, vol. 22, pp. 87–119, Mar. 2014.
- [11] D. S. Rosenblum, “Formal methods and testing: Why the state-of-the-art is not the state-of-the-practice - ISSTA’96/FMSP ’96 panel summary,” *Softw. Eng. Notes*, vol. 21, no. 4, pp. 64–66, Jul. 1996.
- [12] C. Alexander, S. Ishikawa, and M. Silverstein, *A Pattern Language*, ser. Center for Environmental Structure Series, Oxford University Press, 1977, vol. 2.
- [13] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns*, Addison-Wesley, 1995.
- [14] D. Bianculli, C. Ghezzi, C. Pautasso, and P. Senti, “Specification patterns from research to industry: a case study in service-based applications,” in *Proc. 34th Int. Conf. Softw. Eng., Zürich, Switzerland*, IEEE Computer Society Press, Jun. 2012, pp. 992–1000.
- [15] M.-A. Esteve, J.-P. Katoen, V. Y. Nguyen, B. Postma, and Y. Yush-tein, “Formal correctness, safety, dependability, and performance analysis of a satellite,” in *Proc. 34th Int. Conf. Softw. Eng.*, Jun. 2–9, 2012, Zürich, Switzerland, M. Glinz, G. C. Murphy, and M. Pezzè, Eds. IEEE, 2012, pp. 1022–1031.
- [16] A. Post, I. Menzel, J. Hoenicke, and A. Podelski, “Automotive behavioral requirements expressed in a specification pattern system: a case study at bosch,” *Requirements Eng.*, vol. 17, no. 1, pp. 19–33, 2012.
- [17] S. Konrad, “Model-driven development and analysis of high assurance systems,” Ph.D. dissertation, Dept. Comput. Sci. Eng., Michigan State University, East Lansing, MI, USA, 2006.
- [18] M. Lumpe, I. Meedeniya, and L. Grunske, “PSPWizard: Machine-assisted Definition of Temporal Logical Properties with Specification Patterns,” in *Proc. 19th ACM SIGSOFT Symp. Foundations Softw. Eng. and 13th Eur. Softw. Eng. Conf., Tool Demonstrations Track*, Szeged, Hungary, Sep. 2011, pp. 468–471.
- [19] M. Autili, L. Grunske, M. Lumpe, I. Meedeniya, P. Pelliccione, and A. Tang. (2013). Property specification patterns wiki pages. [Online]. Available: <http://ps-patterns.wikidot.com>
- [20] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett, “Property specification patterns for finite-state verification,” in *Proc. Second Workshop Formal Meth. Softw. Prac.*, 1998, pp. 7–15.
- [21] D. O. Paun and M. Chechik, “Events in linear-time properties,” in *Proc. 4th IEEE Int. Symp. Requirements Eng.*, IEEE Computer Society Press, 1999, pp. 123–132.
- [22] R. L. Smith, G. S. Avrunin, L. A. Clarke, and L. J. Osterweil, “Propel: An approach supporting property elucidation,” in *Proc. 24th Int. Conf. Softw. Eng.*, New York, NY, USA: ACM, 2002, pp. 11–21.
- [23] K. Cabrera Castillos, F. Dadeau, J. Juliand, B. Kanso, and S. Taha, “A compositional automata-based semantics for property patterns,” in *Proc. of the 10th Inf. Conf.*, E. B. Johnsen and L. Petre, Eds., vol. LNCS 7940, Heidelberg, Germany: Springer, 2013, pp. 316–330.
- [24] S. Hallé, R. Villemare, and O. Cherkaoui, “Specifying and validating data-aware temporal web service properties,” *IEEE Trans. Softw. Eng.*, vol. 35, no. 5, pp. 669–683, Sep./Oct. 2009.
- [25] F. Bitsch, “Safety patterns—the key to formal specification of safety requirements,” in *Proc. 20th Int. Conf.*, Budapest, Hungary, Sep. 26–28, 2001, U. Voges, Ed., vol. LNCS 2187, Heidelberg, Germany: Springer, 2001, pp. 176–189.
- [26] G. Spanoudakis, C. Kloukinas, and K. Androutsopoulos, “Towards security monitoring patterns,” in *Proc. ACM Symposium on Applied Computing (SAC 2007)*, Seoul, Korea, March 11–15, 2007, Y. Cho, R. L. Wainwright, H. Haddad, S. Y. Shin, and Y. W. Koo, Eds. ACM, 2007, pp. 1518–1525.
- [27] N. G. Leveson, *Safeware—System Safety and Computers*, Addison Wesley, 1995.
- [28] R. Koymans, “Specifying real-time properties with metric temporal logic,” *Real-Time Systems*, vol. 2, no. 4, pp. 255–299, Nov. 1990.
- [29] R. Alur, “Techniques for automatic verification of real-time systems,” Ph.D. dissertation, Dept. Comput. Sci., Stanford Univ., Stanford, CA, USA, Aug. 1991.
- [30] V. Gruhn and R. Laue, “Patterns for timed property specifications,” *Electron. Notes Theoretical Comput. Sci.*, vol. 153, no. 2, pp. 117–133, 2006.
- [31] N. Abid, S. Dal-Zilio, and D. L. Botlan, “Real-time specification patterns and tools,” in *Proc. 17th Int. Workshop*, Paris, France, August 27–28, 2012, M. Stoelinga and R. Pinger, Eds., vol. LNCS 7437, Heidelberg, Germany: Springer, 2012, pp. 1–15.
- [32] A. Aziz, K. Sanwal, V. Singhal, and R. Brayton, “Verifying continuous Yime Markov chains,” in *Proceedings of the 8th International Conference on Computer Aided Verification (CAV ’96)*, R. Alur and T. A. Henzinger, Eds. Springer, Jul. 1996, pp. 269–276.
- [33] R. L. Cobleigh, G. S. Avrunin, and L. A. Clarke, “User guidance for creating precise and accessible property specifications,” in *Proc. 14th ACM SIGSOFT Int. Symp. Foundations of Softw. Eng. (FSE ’06)*, New York, NY, USA: ACM, 2006, pp. 208–218.
- [34] M. Autili, P. Inverardi, and P. Pelliccione, “Graphical scenarios for specifying temporal properties: An automated approach,” *Autom. Softw. Eng.*, vol. 14, no. 3, pp. 293–340, Sep. 2007.
- [35] M. Autili and P. Pelliccione, “Towards a graphical tool for refining user to system requirements,” *Electron. Notes Theoretical Comput. Sci.*, vol. 211, pp. 147–157, 2008.
- [36] P. Zave and M. Jackson, “Four dark corners of requirements engineering,” *ACM Trans. Softw. Eng. Methodol.*, vol. 6, no. 1, pp. 1–30, 1997.
- [37] P. Zhang, L. Grunske, A. Tang, and B. Li, “A formal syntax for probabilistic timed property sequence charts,” in *ASE ’09: Proc. 24th IEEE/ACM Int. Conf. Autom. Softw. Eng.*, IEEE Computer Society, 2009, pp. 500–504.
- [38] P. Zhang, B. Li, and L. Grunske, “Timed property sequence chart,” *J. Syst. Softw.*, vol. 83, no. 3, pp. 371–390, 2010.
- [39] L. D. Guglielmo, F. Fummi, N. Orlandi, and G. Pravadelli, “DDPSL: An easy way of defining properties,” in *Proc. 28th Int. Conf. Comput. Des. (ICCD 2010)*, 3–6 October 2010, Amsterdam, The Netherlands, IEEE, 2010, pp. 468–473.
- [40] *IEEE Standard for Property Specification Language (PSL)*, IEEE Standard 1850–2005, pp. 1–143, 2005.
- [41] G. Holzmann, *The Spin Model Checker: The Primer and Reference Manual*, 1st ed. Addison-Wesley Professional, 2003.
- [42] A. Capozucca, N. Gueffi, and P. Pelliccione, “The fault-tolerant insulin pump therapy,” in *Rigorous Development of Complex Fault-Tolerant Systems*, M. Butler, C. B. Jones, A. Romanovsky, and E. Troubitsyna, Eds. Berlin, Heidelberg: Springer, 2006, pp. 59–79.
- [43] M. Kwiatkowska, G. Norman, and D. Parker, “PRISM 4.0: Verification of probabilistic real-time systems,” in *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV’11)*, G. Gopalakrishnan and S. Qadeer, Eds., vol. LNCS 6806, Heidelberg, Germany: Springer, 2011, pp. 585–591.
- [44] Presto-Consortium, “D2.1: Case studies modelling and design—use case specification,” 2012, http://www.presto-embedded.eu/index.php?option=com_content&task=view&id=31&Itemid=55
- [45] M. Haydar, H. Sahraoui, and A. Petrenko, “Specification patterns for formal web verification,” in *Proc. Eighth Int. Conf. Web Eng.*, Jul. 2008, pp. 240–246.
- [46] R. Alur, T. Feder, and T. A. Henzinger, “The benefits of relaxing punctuality,” *J. ACM*, vol. 43, no. 1, pp. 116–146, Jan. 1996.
- [47] I. Malavolta, H. Muccini, P. Pelliccione, and D. Tamburri, “Providing architectural languages and tools interoperability through model transformation technologies,” *IEEE Trans. Softw. Eng.*, vol. 36, no. 1, pp. 119–140, Jan. 2010.



Marco Autili is currently an assistant professor at the Department of Information Engineering Computer Science and Mathematics (DISIM), University of L'Aquila, L'Aquila, Italy. At DISIM, he is member of the Software Engineering and Architecture group (SEA Lab). His main research interests include software engineering, formal methods, context-oriented programming, and distributed systems. In these areas, main research and development activities include: engineering of service-oriented systems and component-based systems; automated synthesis for composing distributed systems, from coordination to protocol mediation/adaptation; formal specification and analysis of complex distributed systems; formal specification and checking of temporal properties; context-oriented programming and resource-oriented analysis of adaptable (mobile) applications. He actively works on the (from theory to practice) application of formal methods to the verification, analysis and automatic synthesis of complex distributed systems, and application of context-oriented programming and analysis techniques to the development of adaptable (mobile) applications.



Lars Grunske is currently a professor at the University of Stuttgart, Stuttgart, Germany, and heading the Reliable Software Systems (RSS) group at the Institute of Software Technology. He has active research interests in the areas of validation and verification of systems and software. His main focus is on architecture optimization and model-based dependability evaluation of complex software intensive systems.



Markus Lumpe received the Diplom-Ingenieur (Dipl.-Ing.) from Dresden University of Technology, Dresden, Germany, and the PhD degree in computer science from the University of Berne, Berne, Switzerland. He has worked in industry and academia in Germany, Switzerland, and the USA. He is currently a senior lecturer in computer science and software engineering at the Faculty of Science, Engineering and Technology at Swinburne University of Technology, Melbourne, VIC, Australia. His research interests

include the design and implementation of programming languages, software composition, empirical software analysis, formal methods, and software evolution.



Patrizio Pelliccione received the PhD degree at the University of L'Aquila, L'Aquila, Italy, and since February 1, 2014, he has been the Docent in software engineering, the title given by the University of Gothenburg, Gothenburg, Sweden. He is currently an associate professor at the Department of Computer Science and Engineering, Chalmers University of Technology and University of Gothenburg, Sweden, and is also an assistant professor (on leave) at the Department of Information Engineering, Computer Science and Mathematics, University of L'Aquila. His research topics are mainly in software engineering, software architectures modelling and verification, and formal methods. He has co-authored more than 80 publications in journals and international conferences and workshops in this theme. He is the founding member of SERENE-Software Engineering for RESilient systems-ERCIM Working Group. He has been on the program committees for several conferences, and is a reviewer for top journals in the software engineering domain. In his research activity he has collaborated with several industries such as Volvo, Ericsson, Thales Italia, Selex Marconi telecommunications, Siemens, Saab, TERMA, etc.



Antony Tang received the PhD degree in information technology from the Swinburne University of Technology, Melbourne, VIC, Australia. He is currently an associate professor in Swinburne University of Technology's Faculty of Science, Engineering and Technology. Prior to being a researcher, he had spent many years designing and developing software systems. His research interests include software architecture design reasoning, software development processes, software architecture and knowledge engineering. He is a member of the ACM and the IEEE.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**